

Arquitetura Backend Enterprise

Clean Architecture, DDD, Segurança e
Multi-Tenancy com TypeScript

AI Software Engineering Academy

Arquitetura Backend Enterprise

Clean Architecture, DDD, Segurança e Multi-Tenancy com

TypeScript
Autor: Al Software Engineering Academy

Edição: 1 - 2026

Fundamentos de Arquitetura

Módulo 08 -- Arquitetura: Clean Architecture, DDD e SOLID

Pensar antes de programar.

1. Por que arquitetura importa

Arquitetura é a **estrutura fundamental** de um sistema. São as decisões que, se tomadas errado, custam caro para mudar.

O custo de uma arquitetura ruim

[text]

Arquitetura Ruim:

```
Feature nova: 2 semanas  
Por quê? "O código é um macarrão"  
"Toda mudança quebra algo"  
"Melhor reescrever do zero"
```

Arquitetura Boa:

```
Feature nova: 2 dias  
Por quê? "Só adicionar um módulo"  
"Mudei uma coisa sem quebrar nada"  
"Testes garantem que funciona"
```

O que arquitetura define

[text]

Componentes:	Em quais partes o sistema se divide?
Comunicação:	Como as partes se comunicam?
Dados:	Como os dados fluem e são armazenados?
Tecnologia:	Qual stack suporta a estrutura?
Equipe:	Como o time se organiza para desenvolver?

2. SOLID -- Os 5 princípios

SOLID não é uma arquitetura -- é um **conjunto de princípios** que boas arquiteturas seguem.

S -- Single Responsibility Principle

Uma classe deve ter um, e apenas um, motivo para mudar.

[typescript]

```
// Ruim: Service faz tudo
class UserService {
  createUser(data: CreateUserDto) { /* ... */ }
  sendWelcomeEmail(email: string) { /* ... */ }
  generateReport() { /* ... */ }
}

// Bom: Responsabilidades separadas
class CreateUserUseCase {
  execute(data: CreateUserDto) { /* ... */ }
}

class EmailService {
  sendWelcome(user: User) { /* ... */ }
}

class ReportGenerator {
  generate(): Report { /* ... */ }
}
```

O -- Open/Closed Principle

Aberto para extensão, fechado para modificação.

[typescript]

```
// Ruim: Modificar para adicionar tipo
class PaymentProcessor {
  process(type: string, amount: number) {
    if (type === 'credit') { /* ... */ }
    if (type === 'debit') { /* ... */ }
  }
}
```

```
if (type === 'pix') { /* ... */ } // modifiquei!
```

}

}

// Bom: Estender sem modificar

```
interface PaymentMethod {
```

```
process(amount: number): void;
```

}


```
class CreditCardPayment implements PaymentMethod { /* ... */ }
```

```
class PixPayment implements PaymentMethod { /* ... */ }
```


L -- Liskov Substitution Principle

Subtipos devem ser substituíveis por seus tipos base.

```
[typescript]
// Ruim: Quebra a expectativa
class Bird {
  fly(): void { /* voa */ }
}

class Penguin extends Bird {
  fly(): void { throw new Error('Penguins não voam'); } //
}

// Bom: Abstração correta
interface Bird { }
interface FlyingBird extends Bird {
  fly(): void;
}
class Penguin implements Bird { }
class Sparrow implements Bird, FlyingBird { }
```

I -- Interface Segregation Principle

Interfaces específicas são melhores que interfaces genéricas.

```
[typescript]
// Ruim: Interface genérica
interface Worker {
  work(): void;
  eat(): void;
  sleep(): void;
}

// Bom: Interfaces específicas
interface Workable { work(): void; }
interface Eatable { eat(): void; }
interface Sleepable { sleep(): void; }
```

D -- Dependency Inversion Principle

Dependa de abstrações, não de implementações concretas.

[typescript]

```
// Ruim: Dependência concreta
class OrderService {
  private repository = new PrismaOrderRepository(); // concreto
}

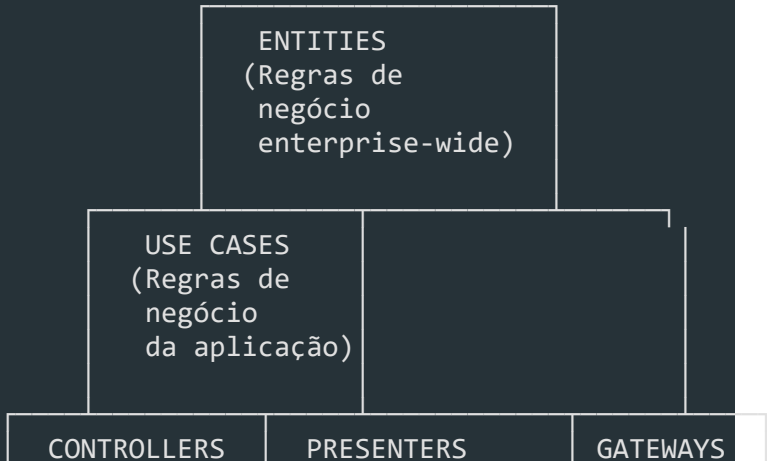
// Bom: Dependência abstrata
class OrderService {
  constructor(
    private repository: OrderRepository // interface
  ) { }
}
```

3. Clean Architecture -- A Regra da Dependência

Clean Architecture é uma arquitetura que organiza o código em **círculos concêntricos**.

As camadas

[text]



| (Adaptadores | (Adaptadores | (Adaptadores |

| de entrada) | de saída) | de saída) |





FRAMEWORKS & DRIVERS

| (NestJS, Prisma, Next.js, PostgreSQL...) |



A Regra da Dependência

*As dependências de código fonte apontam ****para dentro****, em direção ao centro.*

Nada no círculo interno pode saber algo sobre o círculo externo.

[text]

ERRADO: Use Case importa Prisma
UseCase -> PrismaClient (dependência para fora!)

CERTO: Use Case depende de interface
UseCase -> UserRepository (interface)
PrismaUserRepository -> UserRepository (implementa)

O que vai em cada camada

Camada	O que contém	Pode importar
Entities	Regras de negócio Enterprise	Nada externo
Use Cases	Regras de negócio da aplicação	Entities
Controllers/Presenters	Adaptação de entrada/saída	Use Cases
Frameworks	Implementações concretas	Qualquer adaptador

4. DDD -- Domain-Driven Design

DDD é uma abordagem que coloca o **domínio do negócio** no centro do desenvolvimento.

Linguagem Ubíqua

A mesma linguagem usada pelo negócio deve ser usada no código.

[text]

Negócio: "Um cliente pode abrir um chamado"

Código: `client.openTicket(ticket)`

Negócio: "Um cliente pode abrir um ticket de suporte"

Código: `client.createSupportTicket(ticket)` (outra linguagem)

Bounded Contexts

Um Bounded Context é um **limite explícito** onde um modelo de

[text]	
Contexto de Vendas:	Contexto de Logística:
Cliente = quem compra	Cliente = quem recebe
Produto = item no catálogo	Produto = item no estoque
Pedido = carrinho	Pedido = carga para entrega
São modelos DIFERENTES do mesmo conceito!	

Elementos do DDD

[text]	
ENTITY	
-> Tem identidade única (id)	
-> Muda ao longo do tempo	
-> Ex: Usuario, Pedido, Produto	
VALUE OBJECT	
-> Descrito por seus atributos	
-> Imutável	
-> Ex: Endereco, Email, CPF	
AGGREGATE	
-> Grupo de entidades com raiz	
-> Consistência transacional	
-> Ex: Pedido + Itens (Pedido é raiz)	
REPOSITORY	
-> Abstração de persistência	
-> Interface no domínio	
-> Implementação externa	
DOMAIN SERVICE	
-> Regra de negócio sem estado	
-> Opera em múltiplas entidades	

Exemplo prático de DDD

```
[typescript]
// Entidade
class Usuario {
  private id: UsuarioId;
  private email: Email; // Value Object
  private perfil: Perfil;

  constructor(id: UsuarioId, email: Email, perfil: Perfil) {
    this.id = id;
    this.email = email;
    this.perfil = perfil;
  }

  alterarPerfil(novoPerfil: Perfil): void {
    // Regra de negócio
    if (this.perfil.eAdmin() && !novoPerfil.eAdmin()
      && !this.temOutroAdmin()) {
      throw new Error('Deve haver pelo menos um admin');
    }
    this.perfil = novoPerfil;
  }
}

// Value Object
class Email {
  private readonly valor: string;

  constructor(valor: string) {
    if (!valor.includes('@')) {
      throw new Error('Email inválido');
    }
    this.valor = valor;
  }

  toString(): string { return this.valor; }
}

// Repository (interface no domínio)
```

```
interface UsuarioRepository {
```

```
findById(id: UsuarioId): Promise<Usuario | null>;
```

```
save(usuario: Usuario): Promise<void>;
```

```
delete(id: UsuarioId): Promise<void>;
```

}

// Use Case


```
class AlterarPerfilUseCase {
```

constructor(

```
private usuarioRepo: UsuarioRepository,
```

```
private emailService: EmailService
```

) {}


```
async execute(input: { usuarioId: UsuarioId; novoPerfil: Perfil
```

```
const usuario = await this.usuarioRepo.findById(input.usuarioId)
```



```
if (!usuario) throw new NotFoundError('Usuário não encontrado');
```



```
usuario.alterarPerfil(input.novoPerfil);
```

```
await this.usuarioRepo.save(usuario);
```


// Efeito colateral via evento

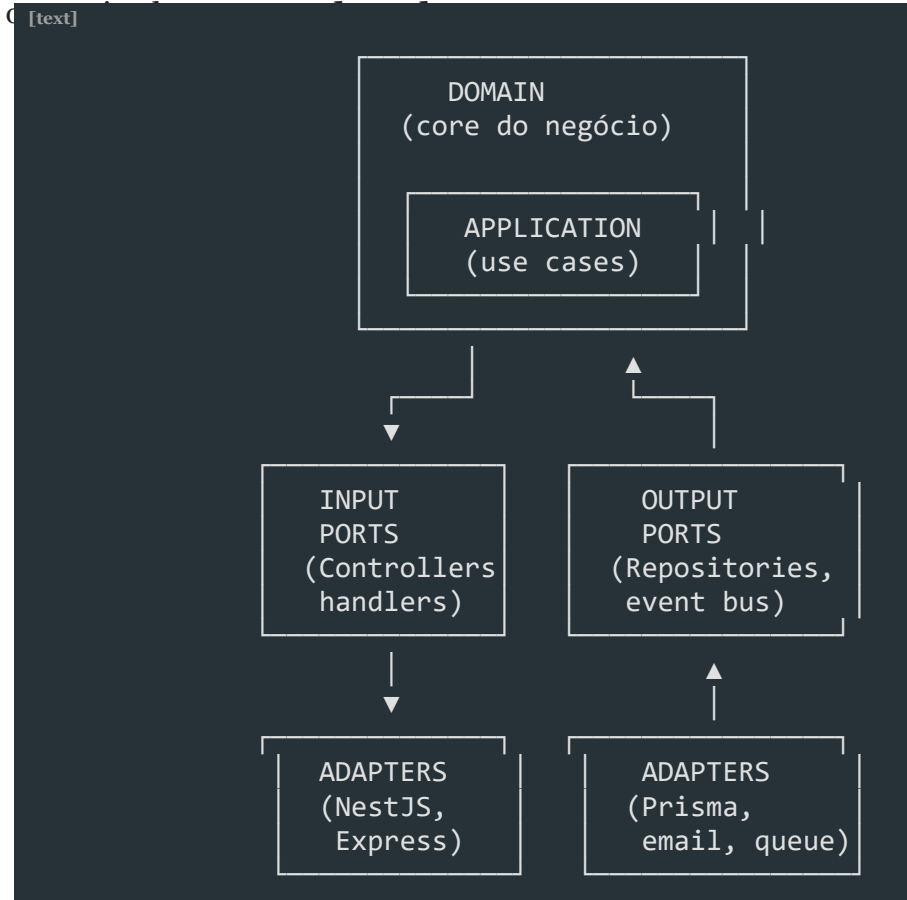
```
await this.emailService.enviarNotificacao(usuario.email);
```

}

}

5. Arquitetura Hexagonal (Ports & Adapters)

A arquitetura hexagonal é uma variação da Clean Architecture que usa



Abaixo, uma visão de container C4 da mesma arquitetura:

[Arquitetura de Referência Backend -

Container](/knowledge-factory/products/courses/arquitetura-backend/module-o8-arquitetura/assets/diagram-c4-container.svg)

Portas

[typescript]

```
// Porta de saída (interface no domínio)
interface PedidoRepository {
```

```
save(pedido: Pedido): Promise<void>;
```

```
findById(id: PedidoId): Promise<Pedido | null>;
```

}

// Porta de entrada (interface do use case)

```
interface CriarPedidoUseCase {
```



```
execute(input: CriarPedidoInput): Promise<CriarPedidoOutput>;
```

}

Adaptadores

```
[typescript]
// Adaptador de saída (implementação externa)
class PrismaPedidoRepository implements PedidoRepository {
  async save(pedido: Pedido): Promise<void> {
    await prisma.pedido.create({
      data: this.toPersistence(pedido),
    });
  }

  private toPersistence(pedido: Pedido): PrismaPedido {
    return {
      id: pedido.id.toString(),
      clienteId: pedido.clienteId.toString(),
      valor: pedido.valor,
    };
  }
}

// Adaptador de entrada (NestJS Controller)
@Controller('/pedidos')
class PedidoController {
  constructor(private criarPedido: CriarPedidoUseCase) {}

  @Post()
  async criar(@Body() body: CriarPedidoDto) {
    return this.criarPedido.execute(this.toInput(body));
  }
}
```

6. Modular Monolith vs Microservices

Modular Monolith

Um monólito **modularizado** -- código em módulos bem definidos,

1 [text]

Prós:

Contras:

- Simplicidade
- Escala tudo junto

- Deploy único
- Ponto único de falha

- Sem latência de rede
- Stack única

- Dados consistentes
- Time grande conflita

- Transações ACID

Quando usar:

- Time pequeno (< 10 devs)

- Sistema com domínios fortemente acoplados

- Startup / MVP

- Quando velocidade > escala

Microservices

S [text]	
Prós:	Contras:
- Escala independente	- Complexidade de rede
- Deploy independente	- Dados distribuídos
- Stack variada	- Consistência eventual
- Times autônomos	- Observabilidade complexa
- Isolamento de falhas	- Testes mais difíceis
Quando usar:	
- Time grande (> 10 devs por serviço)	
- Domínios claramente separados	
- Escala global	
- Times autônomos por domínio	

A recomendação

Comece com Modular Monolith. Extraia microservices quando necessário.

A maioria dos sistemas **não precisa** de microservices. Os que precisam, geralmente começaram como monólito e extraíram os serviços que escalavam de forma diferente.

7. Event-Driven Architecture

Event-Driven Architecture usa **eventos** para comunicação entre componentes.

Conceitos

```
Evento:           "Algo aconteceu"
  -> PedidoCriado, UsuarioCadastrado, PagamentoConfirmado

Produtor:         Quem gera o evento
  -> Serviço de pedidos publica "PedidoCriado"
```


Consumidor: Quem reage ao evento

-> Serviço de email escuta "PedidoCriado" e envia confirmação

Barramento: Meio de transporte

-> RabbitMQ, Kafka, Redis Pub/Sub

Exemplo

```
[typescript]
// Evento
class PedidoCriadoEvent {
  constructor(
    readonly pedidoId: string,
    readonly clienteEmail: string,
    readonly valor: number,
  ) {}
}

// Produtor
class CriarPedidoUseCase {
  constructor(
    private pedidoRepo: PedidoRepository,
    private eventBus: EventBus,
  ) {}

  async execute(input: CriarPedidoInput): Promise<void> {
    const pedido = Pedido.criar(input.clienteId, input.itens);
    await this.pedidoRepo.save(pedido);

    // Publica evento
    await this.eventBus.publish(
      new PedidoCriadoEvent(pedido.id, input.clienteEmail, pedido.valor);
    );
  }
}

// Consumidor
class EmailHandler {
  @OnEvent(PedidoCriadoEvent)
  async enviarConfirmacao(event: PedidoCriadoEvent) {
    await this.emailService.send({
      to: event.clienteEmail,
      subject: 'Pedido confirmado!',
      body: `Pedido #${event.pedidoId} no valor de R$ ${event.valor}`;
    });
  }
}
```

}

8. Aplicando na prática com NestJS + Prisma

Estrutura de pastas seguindo Clean Architecture + DDD

```
src/
├── domain/                                # Círculo mais interno
│   ├── entities/                         # Entidades de domínio
│   │   └── usuario.ts
│   ├── value-objects/                   # Value Objects
│   │   └── email.ts
│   ├── repositories/                   # Interfaces (portas)
│   │   └── usuario.repository.ts
│   ├── services/                       # Domain Services
│   │   └── autenticacao.service.ts
│
├── application/                         # Use Cases
│   ├── use-cases/
│   │   └── criar-usuario.use-case.ts
│   ├── dto/
│   │   └── criar-usuario.dto.ts
│
├── infrastructure/                     # Adaptadores (implementações)
│   ├── persistence/
│   │   ├── prisma/
│   │   │   ├── prisma.service.ts
│   │   │   └── repositories/
│   │   │       └── prisma-usuario.repository.ts
│   ├── messaging/
│   │   └── rabbitmq-event-bus.ts
│   ├── email/
│   │   └── nodemailer-email.service.ts
│
├── presentation/                       # Adaptadores de entrada
│   ├── controllers/
│   │   └── usuario.controller.ts
│   ├── guards/
│   │   └── jwt-auth.guard.ts
```


└─ main.ts

Modelagem de Sistemas

Módulo 09 -- Modelagem de Dados: Prisma e PostgreSQL

****A base de todo sistema Enterprise.****

- - -

1. Por que modelagem importa

Modelagem de dados é a ****fundação**** do sistema. Erros aqui são

O custo de uma modelagem ruim

Modelagem ruim:

┌	
└	
	Tabela sem índices -> query lenta
	Relação errada -> dados inconsistentes
	Falta de soft delete -> perda de dados
	Sem audit trail -> impossível auditar
	Migração corretiva -> horas de trabalho
└	

Modelagem boa:

┌	
└	
	Índices certos -> queries rápidas
	Relações corretas -> dados íntegros
	Constraints -> validade dos dados
	Migrações testadas -> sem surpresas
└	

```
---  
  
## 2. Entidades e Relacionamentos  
  
### Tipos de Relacionamento
```

1:1 -- Um usuário tem um perfil

1:N -- Um usuário tem muitos pedidos

```
┌
```

Exemplo no Prisma


```
// 1:1
model User {
  id    String @id @default(cuid())
  email String @unique
  profile Profile?
}
model Profile {
  id    String @id @default(cuid())
  fullName String
  avatarUrl String?
  userId String @unique
  user   User @relation(fields: [userId], references: [id])
}
// 1:N
model Order {
  id    String @id @default(cuid())
  total Decimal
  userId String
  user  User @relation(fields: [userId], references: [id])
  items OrderItem[]
  createdAt DateTime @default(now())
}
model OrderItem {
  id    String @id @default(cuid())
  orderId String
  order  Order @relation(fields: [orderId], references: [id])
  productId String
  product Product @relation(fields: [productId], references: [id])
  quantity Int
  price   Decimal
}
// N:M
model Product {
  id    String @id @default(cuid())
```

```
name      String
categories ProductCategory[]
}
model Category {
  id      String      @id @default(cuid())
  name    String
  products ProductCategory[]
}
model ProductCategory {
  productId String
  categoryId String
  product   Product @relation(fields: [productId], references: [id])
  category_ Category @relation(fields: [categoryId], references: [id])
  @@create([productId, categoryId])
}
```

```
[text]

---

## 3. Soft Delete e Audit Trail

### Soft Delete

Nunca delete dados definitivamente em sistemas Enterprise.
```

```
model User {
  id      String @id @default(cuid())
  email   String @unique
  name    String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime? // Soft delete
  // Filtro global no Prisma
  @@where("@deletedAt is null")
}

// Service
```



```
class UserService {
  async softDelete(id: string): Promise<void> {
    await this.prisma.user.update({
      where: { id },
      data: { deletedAt: new Date() },
    });
  }
  async findAll(): Promise<User[]> {
    // O @@where garante que deleted não apareça
    return this.prisma.user.findMany();
  }
}
```

```
} [text]
```

```
### Audit Trail
```

```
model AuditLog {
  id      String @id @default(cuid())
  entity   String // "User", "Order", "Product"
  entityId String // ID do registro
  action   String // "CREATE", "UPDATE", "DELETE"
  changes  Json?  // { "before": {...}, "after": {...} }
  userId   String?
  ip       String?
  userAgent String?
  createdAt DateTime @default(now())
  @@index([entity, entityId])
  @@index([userId])
  @@index([createdAt])
}
```

```
// AuditService
```

```
class AuditService {
  async log(input: AuditInput): Promise<void> {
```

```
await this.prisma.auditLog.create({
  data: {
    entity: input.entity,
    entityId: input.entityId,
    action: input.action,
    changes: JSON.parse(JSON.stringify(input.changes)),
    userId: input.userId,
    ip: input.ip,
    userAgent: input.userAgent,
  },
});
}
} // Uso com hook do Prisma
prisma.$use(async (params, next) => {
  const result = await next(params);
  if ([ 'create', 'update', 'delete' ].includes(params.action)) {
    await auditService.log({
      entity: params.model,
      entityId: result?.id,
      action: params.action.toUpperCase(),
      changes: params.args.data,
    });
  }
  return result;
});
} [text]

---

## 4. Índices e Performance

### Quando criar índices

model Order {
```

```
id      String @id @default(cuid())
userId  String
status  OrderStatus
total   Decimal
createdAt DateTime @default(now())
// Índice para busca por usuário (foreign key)
@@index([userId])
// Índice composto para filtro comum
@@index([userId, status])
// Índice para ordenação por data
@@index([createdAt])
// Índice parcial para pedidos ativos
@@index([status, createdAt])
}
```

Regras de índices

Crie índices para:

- Foreign keys (toda FK deve ter índice)
- Campos usados em WHERE
- Campos usados em ORDER BY
- Campos usados em JOIN

Evite:

- Índices em colunas de baixa cardinalidade (boolean)
- Muitos índices em tabelas pequenas (< 1000 registros)

Query Performance

```
// N+1 -- busca em loop
const orders = await prisma.order.findMany();
for (const order of orders) {
  const items = await prisma.orderItem.findMany({
    where: { orderId: order.id },
  });
}
// Eager loading -- busca tudo de uma vez
```

```
const orders = await prisma.order.findMany({
  include: {
    items: true,
    user: true,
  },
})
```

5. Migrações Seguras

Criando migrações

Criar migration baseada no schema

`npx prisma migrate dev --name create-user-table`

Aplicar em produção

`npx prisma migrate deploy`

Resetar banco (dev)

```
1
  ### Migrações sem downtime

// Ruim: renomear coluna diretamente (quebra queries em
execução)
model User {
  fullname String // Antigo
  name   String // Novo -- erro se ambas existirem
}
// Bom: expand-migrate-contract
// Passo 1: Adicionar nova coluna (sem remover antiga)
model User {
  fullname String? // Nullable agora
  name   String?
}
// Passo 2: Preencher dados (script separado)
await prisma.user.updateMany({
  where: { name: null },
  data: { name: prisma.user.fullname }, // valor do campo antigo
})
// Passo 3: Remover coluna antiga (próxima release)
model User {
  name String // Único campo
}
[ ] [text]
```

6. Estratégias de Backup

Tipos de backup

Full: Cópia completa do banco

Quando: Diário

Tempo: Lento, ocupa espaço

Restore: Completo, mais simples

Incremental: Apenas mudanças desde o último backup

Quando: Horário

Tempo: Rápido, ocupa pouco espaço

Restore: Precisa do full + todos incrementais

WAL (Write-Ahead Log): Log de transações

Quando: Contínuo

```
### Script de backup
```

```
#!/bin/bash
```


backup.sh -- backup PostgreSQL com compressão

```
DB_NAME="app"
DB_USER="admin"
BACKUP_DIR="/backups"
DATE=$(date +%Y%m%d_%H%M%S)
pg_dump -U $DB_USER -d $DB_NAME \
--format=custom \
--compress=9 \
--file="$BACKUP_DIR/$DB_NAME-$DATE.dump"
```

Manter apenas últimos 7 dias

```
find $BACKUP_DIR -name "*dump" -mtime +7 -delete  
6  
### Restore
```

Restore completo

```
pg_restore -U $DB_USER -d $DB_NAME \
--clean \
--if-exists \
```

```
[text]
```

```
---
```

```
## 7. Schema completo de exemplo
```

O diagrama abaixo resume as entidades, atributos e cardinalidades

![Modelo Entidade-Relacionamento Enterprise](/knowledge-factory)

```
generator client {
  provider = "prisma-client-js"
}
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
enum UserRole {
  ADMIN
  MANAGER
  USER
}
enum OrderStatus {
  PENDING
  CONFIRMED
  SHIPPED
  DELIVERED
  CANCELLED
}
model Tenant {
```

```

id      String  @id @default(cuid())
slug    String  @unique
name     String
users    User[]
createdAt DateTime @default(now())
@@map("tenants")
}
model User {
  id      String  @id @default(cuid())
  email    String  @unique
  name     String
  password String
  role     UserRole @default(USER)
  tenantId String
  tenant   Tenant  @relation(fields: [tenantId], references: [id])
  orders   Order[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime?
  @@index([tenantId])
  @@index([email])
  @@index([tenantId, role])
  @@map("users")
}
model Product {
  id      String  @id @default(cuid())
  sku     String  @unique
  name     String
  description String?
  price    Decimal @db.Decimal(10, 2)
  stock    Int     @default(0)
  active   Boolean @default(true)
  tenantId String
  orderItems OrderItem[]

```

```

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt
    deletedAt DateTime?
    @@index([tenantId])
    @@index([sku])
    @@index([active])
    @@map("products")
}
model Order {
  id String @id @default(cuid())
  total Decimal @db.Decimal(10, 2)
  status OrderStatus @default(PENDING)
  userId String
  user User @relation(fields: [userId], references: [id])
  items OrderItem[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  @@index([userId])
  @@index([status])
  @@index([createdAt])
  @@map("orders")
}
model OrderItem {
  id String @id @default(cuid())
  orderId String
  order Order @relation(fields: [orderId], references: [id])
  productId String
  product Product @relation(fields: [productId], references: [id])
  quantity Int
  price Decimal @db.Decimal(10, 2)
  @@index([orderId])
  @@index([productId])
  @@map("order_items")
}
model AuditLog {

```

```

id    String @id @default(cuid())
entity String
entityId String
action String
changes Json?
userId String?
ip    String?
createdAt DateTime @default(now())
@@index([entity, entityId])
@@index([userId])
@@index([createdAt])
@@map("audit_logs")
}

```

```
---
```

Resumo

1. ****Modelagem é a fundação**** -- erros aqui são os mais caros
2. ****1:1, 1:N, N:M**** -- conheça os 3 tipos de relacionamento
3. ****Soft Delete**** -- nunca delete dados (deletedAt)
4. ****Audit Trail**** -- toda ação importante deve ser registrada
5. ****Índices**** -- toda FK precisa de índice; índices compostos
6. ****Eager Loading**** -- previne N+1
7. ****Migrações seguras**** -- expand-migrate-contract para mudanças
8. ****Backup**** -- full + incremental + WAL; testar restore periódico

Desenvolvimento Backend

Módulo 10 -- Backend: APIs Enterprise com NestJS

****Construindo backends robustos, testáveis e seguros.****

```
---
```

1. Por que NestJS?

NestJS é o framework Node.js mais adequado para sistemas Enterprise

Comparação

| Característica | Express | Fastify | NestJS |



| Arquitetura | Livre | Livre | Modular (módulos, controllers,

| TypeScript | Opcional | Opcional | Nativo (decorators, metadados)

| [Guards/Interceptors](#) | [Manual](#) | [Manual](#) | [Nativo \(auth, validaç](#)

| [OpenAPI/Swagger](#) | [Manual](#) | [Manual](#) | [Nativo \(@nestjs/swagger\)](#)

| Testabilidade | Média | Média | Alta (DI + módulos testáveis)

| Ecossistema Enterprise | Baixo | Baixo | Alto (Guard, Interce

Módulo NestJS típico


```
@Module({
  imports: [PrismaModule, HttpModule],
  controllers: [UserController],
  providers: [UserService, UserRepository, JwtStrategy],
  exports: [UserService],
})
```

```
6 [text]
```

```
! [Arquitetura de Referencia Backend] (/knowledge-factory/product
```

```
---
```

```
## 2. Estrutura de módulos
```

```
### Organização por domínio
```

```
src/
```

```
— modules/
```

```
| — users/
```

```
| | — user.module.ts
```

```
| | — user.controller.ts
```

```
| | — user.service.ts
```

```
| | — user.repository.ts
```

```
| | — dto/
```

```
| | | — create-user.dto.ts
```

```
| | | — update-user.dto.ts
```

```
| | — entities/
```

```
| | — user.entity.ts
```

```
| — orders/
```

```
| | — ...
```

```
| — payments/
```

```
| | — ...
```

```
— common/
```

```
| — guards/  
| — interceptors/  
| — pipes/  
|   └─ filters/  
— config/  
|   └─ app.config.ts
```

```
### Por que essa estrutura?
```

Separação por domínio:

- > Tudo relacionado a "users" está junto
- > Fácil de navegar e manter
- > Cada módulo é autocontido

Módulo raiz (AppModule).

- > Importa os módulos de domínio
- > Tempo de inicialização mais rápido

```
---
```

```
## 3. Controllers, Services, Repositories
```

```
### Camadas
```

Controller (Rota)

chamada

Service (Lógica de negócio)

chamada

Repository (Persistência)

```
### Controller
```

```

@Controller('users')
@UseGuards(JwtAuthGuard)
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Post()
  @ApiOperation({ summary: 'Criar usuário' })
  @ApiResponse({ status: 201, type: UserResponse })
  async create(@Body() dto: CreateUserDto) {
    return this.userService.create(dto);
  }

  @Get(':id')
  async findOne(@Param('id', ParseUUIDPipe) id: string) {
    return this.userService.findById(id);
  }

  @Get()
  async findAll(
    @Query('page', ParseIntPipe) page: number = 1,
    @Query('limit', ParseIntPipe) limit: number = 10,
  ) {
    return this.userService.findAll({ page, limit });
  }

  @Patch(':id')
  async update(
    @Param('id', ParseUUIDPipe) id: string,
    @Body() dto: UpdateUserDto,
  ) {
    return this.userService.update(id, dto);
  }

  @Delete(':id')
  @HttpCode(HttpStatus.NO_CONTENT)
  async remove(@Param('id', ParseUUIDPipe) id: string) {
    await this.userService.softDelete(id);
  }
}

### Service

```



```

@Injectable()
export class UserService {
  constructor(
    private readonly userRepo: UserRepository,
    private readonly emailService: EmailService,
  ) {}
  async create(dto: CreateUserDto): Promise<UserResponse> {
    const email = new Email(dto.email);
    const exists = await this.userRepo.findByEmail(email.toString());
    if (exists) throw new ConflictException('Email já cadastrado');
    const hashedPassword = await bcrypt.hash(dto.password, 12);
    const user = User.create({
      name: dto.name,
      email: email,
      password: hashedPassword,
    });
    await this.userRepo.save(user);
    await this.emailService.sendWelcome(user.email.toString());
    return UserResponse.from(user);
  }
  async findById(id: string): Promise<UserResponse> {
    const user = await this.userRepo.findById(id);
    if (!user) throw new NotFoundException('Usuário não encontrado');
    return UserResponse.from(user);
  }
  async findAll(pagination: PaginationInput):
    Promise<PaginatedResult<UserResponse>> {
    return this.userRepo.findAll(pagination);
  }
}

```

```

### Repository

```

```

@Injectable()
export class UserRepository {

```

```

constructor(private readonly prisma: PrismaService) {}

async findById(id: string): Promise<User | null> {
  const raw = await this.prisma.user.findUnique({ where: { id } });
  return raw ? this.toDomain(raw) : null;
}

async findByEmail(email: string): Promise<User | null> {
  const raw = await this.prisma.user.findUnique({ where: { email } });
  return raw ? this.toDomain(raw) : null;
}

async findAll(pagination: PaginationInput):
Promise<PaginatedResult<User>> {
  const [items, total] = await Promise.all([
    this.prisma.user.findMany({
      skip: (pagination.page - 1) * pagination.limit,
      take: pagination.limit,
      orderBy: { createdAt: 'desc' },
    }),
    this.prisma.user.count(),
  ]);
  return {
    items: items.map(this.toDomain),
    total,
    page: pagination.page,
    limit: pagination.limit,
  };
}

async save(user: User): Promise<void> {
  await this.prisma.user.create({ data: this.toPersistence(user) });
}

async update(id: string, user: User): Promise<void> {
  await this.prisma.user.update({
    where: { id },
    data: this.toPersistence(user),
  });
}

async softDelete(id: string): Promise<void> {

```

```
await this.prisma.user.update({
  where: { id },
  data: { deletedAt: new Date() },
});

private toDomain(raw: PrismaUser): User {
  return User.restore(
    raw.id,
    raw.name,
    Email.restore(raw.email),
    raw.password,
    raw.role as UserRole,
  );
}

private toPersistence(user: User): PrismaUserCreateInput {
  return {
    id: user.id,
    name: user.name,
    email: user.email.toString(),
    password: user.password,
    role: user.role,
  };
}
}
```

```
---
```

```
## 4. DTOs e Validação com Zod
```

```
### DTOs (Data Transfer Objects)
```

```
import { z } from 'zod';
export const CreateUserSchema = z.object({
  name: z.string().min(3).max(100),
  email: z.string().email(),
```

```

password: z.string().min(8).regex(
  /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/,
  'Senha deve conter maiúscula, minúscula e número'
),
role: z.enum(['admin', 'manager', 'user']).default('user'),
});
export type CreateUserDto = z.infer<typeof CreateUserSchema>;
export class CreateUserPipe implements PipeTransform {
  transform(value: unknown) {
    const result = CreateUserSchema.safeParse(value);
    if (!result.success) {
      throw new BadRequestException({
        message: 'Dados inválidos',
        errors: result.error.flatten().fieldErrors,
      });
    }
    return result.data;
  }
}

```

```

### Uso no controller

```

```

@Post()
async create(@Body(new CreateUserPipe()) dto: CreateUserDto) {
  return this.userService.create(dto);
}

```

```

### Validação vs Sanitização

```

```

// Validação: rejeitar dados inválidos
password: z.string().min(8)
// Sanitização: transformar dados
email: z.string().email().transform(v => v.toLowerCase()),

```

```

I [text]

```


--

5. Tratamento de Erros

Exception Filters


```
@Catch()
```

```
export class GlobalExceptionHandler implements ExceptionFilter {
```

```
  catch(exception: unknown, host: ArgumentsHost) {
```

```
    const ctx = host.switchToHttp();
```

```
    const response = ctx.getResponse<Response>();
```

```
    if (exception instanceof DomainError) {
```

```
      return response.status(400).json({
```

```
        statusCode: 400,
```

```
        error: exception.code,
```

```
        message: exception.message,
```

```
      });
```

```
    } if (exception instanceof HttpException) {
```

```
      return response.status(exception.getStatus()).json({
```

```
        statusCode: exception.getStatus(),
```

```
        error: exception.name,
```

```
        message: exception.message,
```

```
      });
```

```
  } // Erro não mapeado -- log e retorno genérico
```

```
    console.error('Erro não tratado:', exception);
```

```
    return response.status(500).json({
```

```
      statusCode: 500,
```

```
      error: 'Internal Server Error',
```

```
      message: 'Erro interno do servidor',
```

```
    });
```

```
  }
```

```
}
```

```
### Domain Errors
```

```
export abstract class DomainError extends Error {
```

```
  abstract readonly code: string;
```

```
} export class EmailAlreadyExistsError extends DomainError {
```



```

readonly code = 'EMAIL_ALREADY_EXISTS';
constructor() { super('Email já cadastrado'); }
}
export class InsufficientBalanceError extends DomainError {
  readonly code = 'INSUFFICIENT_BALANCE';
  constructor() { super('Saldo insuficiente'); }
}
export class InvalidEmailError extends DomainError {
  readonly code = 'INVALID_EMAIL';
  constructor() { super('Formato de email inválido'); }
}
} [text]

```

```

---

```

6. Interceptors e Guards

Interceptors (transformação de resposta)

```

@Injectable()
export class TransformInterceptor<T> implements
NestInterceptor<T, ApiResponse<T>> {
  intercept(context: ExecutionContext, next: CallHandler):
Observable<ApiResponse<T>> {
    return next.handle().pipe(
      map(data => ({
        success: true,
        data,
        timestamp: new Date().toISOString(),
      })),
    );
  }
}
// Uso global

```

```

@

```

Guards (proteção de rotas)


```

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly requiredRoles: string[]) {}
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    const user = request.user;
    if (!user) return false;
    return this.requiredRoles.includes(user.role);
  }
}
// Uso no controller
@Delete('/:id')
@UseGuards(new RolesGuard(['admin']))
async remove(@Param('id') id: string) {
  return this.userService.softDelete(id);
}

```

```

} [text]

---

## 7. Paginação, Filtros e Ordenação

### Paginação cursor-based (recomendada para scale)

```

```

interface CursorPaginationInput {
  cursor?: string; // ID do último item
  limit: number; // Itens por página
}
interface CursorPaginatedResult<T> {
  items: T[];
  nextCursor: string | null;
  hasMore: boolean;
}
// Implementação
async findAll(input: CursorPaginationInput):
Promise<CursorPaginatedResult<User>> {
  const items = await this.prisma.user.findMany({

```

```

    take: input.limit + 1, // Pega um a mais para saber se tem próxima
    cursor: input.cursor ? { id: input.cursor } : undefined,
    orderBy: { createdAt: 'desc' },
  });
  const hasMore = items.length > input.limit;
  if (hasMore) items.pop();
  return {
    items: items.map(this.toDomain),
    nextCursor: hasMore ? items[items.length - 1].id : null,
    hasMore,
  };
}

```

```
---
```

```
## 8. Cache com Redis
```

```
### Cache de respostas
```

```

@Injectable()
export class CacheService {
  constructor(private readonly redis: Redis) {}
  async getOrSet<T>(key: string, ttl: number, fetcher: () =>
    Promise<T>): Promise<T> {
    const cached = await this.redis.get(key);
    if (cached) return JSON.parse(cached);
    const data = await fetcher(),
    await this.redis.set(key, JSON.stringify(data), 'EX', ttl);
    return data;
  }
  async invalidate(pattern: string): Promise<void> {
    const keys = await this.redis.keys(pattern);
    if (keys.length > 0) await this.redis.del(keys);
  }
}
// Uso
async findById(id: string): Promise<UserResponse> {

```

```

return this.cacheService.getOrSet(
  user:${id},
  300, // 5 minutos
  () => this.fetchUser(id),
);
} [text]

```

```

---

```

9. Health Checks

```

@Controller('health')
export class HealthController {
  constructor(
    private prisma: PrismaService,
    private redis: RedisService,
  ) {}

  @Get()
  async check() {
    const checks = await Promise.all([
      this.checkDatabase(),
      this.checkRedis(),
    ]);
    const allHealthy = checks.every(c => c.status === 'healthy');
    return {
      status: allHealthy ? 'healthy' : 'degraded',
      timestamp: new Date().toISOString(),
      checks,
    };
  }

  private async checkDatabase() {
    try {
      await this.prisma.$queryRaw`SELECT 1`;
      return { name: 'database', status: 'healthy' };
    } catch {

```

```

    return { name: 'database', status: 'unhealthy' };
  }
  private async checkRedis() {
    try {
      await this.redis.ping();
      return { name: 'redis', status: 'healthy' };
    } catch {
      return { name: 'redis', status: 'unhealthy' };
    }
  }
}

```

```
---
```

10. Testes

```

// Teste de service
describe('UserService', () => {
  let service: UserService;
  let repo: jest.Mocked<UserRepository>;
  let emailService: jest.Mocked<EmailService>;
  beforeEach(async () => {
    repo = { findById: jest.fn(), save: jest.fn() } as any;
    emailService = { sendWelcome: jest.fn() } as any;
    service = new UserService(repo, emailService);
  });
  describe('create', () => {
    it('deve criar usuário com sucesso', async () => {
      const dto = { name: 'João', email: 'joao@email.com', password:
'Senha123' };
      repo.findById.mockResolvedValue(null);
      expect(repo.save).toHaveBeenCalled();
      expect(emailService.sendWelcome).toHaveBeenCalled();
      expect(result).toBeDefined();
    });
  });
});

```

```
    });  
    it('deve lançar erro se email já existe', async () => {  
      await repo.findByEmail.mockResolvedValue(createUser());  
      expect(service.create(mockDto)).rejects.toThrow(ConflictException);  
    });  
  });  
});
```


Segurança

Módulo 12 -- Segurança

Protegendo sistemas Enterprise contra ameaças reais.

1. Por que segurança é o requisito mais importante

Segurança não é uma feature -- é um **pré-requisito**. Um sistema inseguro é um passivo, não um ativo.

O custo de uma falha de segurança

[text]

Falha de segurança típica:

```
Dados vazados
Multas regulatórias (LGPD: até 2% do fat.) |
Perda de confiança dos clientes
Custos de remediação (R$ 200k-2M médio)
Tempo de inatividade
```

Custo de prevenir:

```
Treinamento do time: R$ 5k
Ferramentas de segurança: R$ 1k/mês
Revisão de código: parte do processo
```

Mindset de segurança

```
"Segurança é problema do DevOps"
"Depois a gente adiciona segurança"
"Ninguém vai atacar nosso sistema"

"Segurança é responsabilidade de todos"
"Segurança é parte da definição de 'pronto'"
"Se tem valor, vai ser atacado"
```

2. OWASP Top 10 (2021)

As 10 vulnerabilidades mais críticas em aplicações web.

1. Broken Access Control

Usuário acessa recursos que não deveria.

[typescript]

```
// Ruim: Não verifica se o recurso pertence ao usuário
@Get('/orders/:id')
async getOrder(@Param('id') id: string) {
  return this.orderRepo.findById(id); // qualquer um vê qualquer coisa

// Bom: Verifica propriedade
@Get('/orders/:id')
async getOrder(@Param('id') id: string, @Req() req) {
  const order = await this.orderRepo.findById(id);
  if (order.userId !== req.user.id) {
    throw new ForbiddenException();
  }
  return order;
}
```

2. Cryptographic Failures

Dados sensíveis expostos ou mal protegidos.

[typescript]

```
// Ruim: Senha em texto puro
const user = await prisma.user.create({
  data: { password: req.body.password } // senha em texto puro
});

// Bom: Hash com bcrypt
const hashedPassword = await bcrypt.hash(req.body.password, 12);
const user = await prisma.user.create({
  data: { password: hashedPassword }
});
```

3. Injection

SQL, NoSQL, OS command injection.

[typescript]

```
// Ruim: Query concatenada (SQL injection!)
const users = await prisma.$queryRawUnsafe(
  `SELECT * FROM users WHERE email = '${email}'`
);

// Bom: Query parametrizada (Prisma previne injection)
const user = await prisma.user.findUnique({
  where: { email }
});
```

4. Insecure Design

Falhas no design que permitem ataques.

[typescript]

```
// Ruim: Rate limit não implementado
@Post('/login')
async login(@Body() dto: LoginDto) {
  // Tentativas ilimitadas -- brute force!
}

// Bom: Rate limit com throttling
@Throttle(5, 60) // 5 tentativas por minuto
@Post('/login')
async login(@Body() dto: LoginDto) {
  // ...
}
```

5. Security Misconfiguration

Configurações padrão inseguras.

[typescript]

```
// Ruim: CORS aberto
app.enableCors(); // permite qualquer origem!
```



```
// Bom: CORS restrito
```

```
app.enableCors({
```

```
origin: ['https://app.meusistema.com'],
```



```
methods: ['GET', 'POST', 'PUT', 'DELETE'],
```

});

6-10: Vulnerable Components, Auth Failures, Data Integrity, Logging, SSRF

#	Vulnerabilidade	Prevenção
6	Vulnerable Components	npm audit, Dependabot, manter deps atualizadas
7	Identification/Auth Failures	JWT com expiração, MFA, session segura
8	Data Integrity Failures	Assinatura de dados, validação, CSP
9	Security Logging/Monitoring	Logs estruturados, alertas de segurança
10	SSRF	Validar URLs, restringir rede interna

3. Autenticação com JWT

Fluxo completo

[text]

1. Usuário envia email + senha
2. Servidor valida credenciais
3. Servidor gera ACCESS TOKEN (15min) + REFRESH TOKEN (7d)
4. Cliente armazena e envia access token em requisições
5. Servidor valida token em cada requisição (AuthGuard)
6. Quando access token expira, cliente usa refresh token para obter novo access token

Implementação

[typescript]

```
// auth.service.ts
@Injectable()
export class AuthService {
  constructor(
```

```
private userRepo: UserRepository,
```

```
private JwtService: JwtService,
```

) {}


```
async login(dto: LoginDto): Promise<AuthTokens> {
```



```
const user = await this.userRepo.findByEmail(dto.email);
```

```
if (!user) throw new UnauthorizedException('Credenciais inv
```



```
const passwordValid = await bcrypt.compare(dto.password, us
```

```
if (!passwordValid) throw new UnauthorizedException('Creden
```



```
return this.generateTokens(user);
```

}


```
private async generateTokens(user: User): Promise<AuthTokens>
```

```
const payload = { sub: user.id, email: user.email, role: us
```



```
const accessToken = await this.jwtService.signAsync(payload)
```

```
expiresIn: '15m',
```

});


```
const refreshToken = await this.jwtService.signAsync(payload)
```

```
expiresIn: '7d',
```

```
secret: process.env.JWT_REFRESH_SECRET,
```

});


```
// Armazenar refresh token (para revogação)
```

```
await this.tokenRepo.save(user.id, refreshToken);
```



```
return { accessToken, refreshToken };
```

}


```
async refresh(refreshToken: string): Promise<AuthTokens> {
```

```
try {
```

```
const payload = await this.jwtService.verifyAsync(refresh
```

```
secret: process.env.JWT_REFRESH_SECRET,
```

```
});
```



```
// Verificar se token ainda é válido (não foi revogado)
```

```
const stored = await this.tokenRepo.find(payload.sub, ref
```

```
if (!stored) throw new UnauthorizedException('Token revoked');
```


// Revogar token antigo (rotação)

```
await this.tokenRepo.delete(payload.sub, refreshToken);
```



```
const user = await this.userRepo.findById(payload.sub);
```

```
return this.generateTokens(user);
```

```
} catch {
```

```
throw new UnauthorizedException('Refresh token inválido');
```

}

}


```
async logout(userId: string): Promise<void> {
```



```
await this.tokenRepo.deleteAll(userId);
```

}

}


```
// jwt-auth.guard.ts
```

```
@Injectable()
```

```
export class JwtAuthGuard extends AuthGuard('jwt') {}
```



```
// jwt.strategy.ts
```

@Injectable()

```
export class JwtStrategy extends PassportStrategy(Strategy) {
```

```
constructor() {
```

```
super({
```

```
jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
```

```
secretOrKey: process.env.JWT_SECRET,
```

```
});
```


}


```
async validate(payload: JwtPayload): Promise<User> {
```

```
return { id: payload.sub, email: payload.email, role: payload.role }
```

}

}

4. Autorização com CASL (RBAC)

Definição de abilities

```
[typescript]
// abilities.ts
export type Action = 'manage' | 'create' | 'read' | 'update' | 'delete'
export type Subject = 'User' | 'Order' | 'Product' | 'Report'

export function defineAbilitiesFor(user: User): PureAbility {
  return AbilityBuilder.define((can, cannot) => {
    // Admin pode fazer tudo
    if (user.role === 'admin') {
      can('manage', 'all');
      return;
    }

    // Usuário comum
    can('read', 'User', { id: user.id }); // só próprio p
    can('read', 'Order', { userId: user.id }); // só seus pedi
    can('create', 'Order'); // criar pedido
    can('update', 'Order', { userId: user.id, status: 'pending' });

    // Gerente pode ver relatórios
    if (user.role === 'manager') {
      can('read', 'Report');
      can('read', 'User', { tenantId: user.tenantId });
    }

    // Ninguém pode deletar
    cannot('delete', 'all');
  });
}
```

Uso no controller

```
[typescript]
@Post('/orders')
```

```
async create(@Body() dto: CreateOrderDto, @Req() req) {
```



```
const ability = defineAbilitiesFor(req.user);
```

```
ForbiddenError.from(ability).throwUnlessCan('create', 'Order')
```

```
return this.orderService.create(dto, req.user.id);
```

}


```
@Delete('/:id')
```

```
async delete(@Param('id') id: string, @Req() req) {
```

```
const ability = defineAbilitiesFor(req.user);
```



```
ForbiddenError.from(ability).throwUnlessCan('delete', 'Order')
```

```
// Nunca chega aqui -- admin também não pode deletar
```

}

5. Rate Limiting

Protege contra brute force, DDoS e abuso.

Implementação com @nestjs/throttler

```
[typescript]
// app.module.ts
@Module({
  imports: [
    ThrottlerModule.forRoot([
      {
        ttl: 60000,      // 1 minuto
        limit: 60,       // 60 requisições por minuto (global)
      }
    ]),
  ],
})

// Uso em endpoints específicos
@Throttle(5, 60) // 5 tentativas por minuto
@Post('/login')
async login(@Body() dto: LoginDto) {
  // ...
}

@Throttle(3, 60) // 3 tentativas por minuto
@Post('/password-reset')
async requestPasswordReset(@Body() dto: ResetDto) {
  // ...
}
```

Estratégias adicionais

```
[text]
Rate Limiting por:
  IP:          limitar por endereço IP
  Usuário:     limitar por user ID
  Rota:        limites diferentes por endpoint
  Global:      limite total de requisições

Respostas:
```

429 Too Many Requests

Header: Retry-After: X segundos

6. Headers de Segurança (Helmet + CSP)

Helmet (NestJS)

```
[typescript]
import helmet from 'helmet';

app.use(helmet());

// Configura automaticamente:
// Content-Security-Policy
// X-Content-Type-Options: nosniff
// X-Frame-Options: DENY
// X-XSS-Protection: 0
// Strict-Transport-Security
// Referrer-Policy
```

CSP (Content Security Policy)

```
[typescript]
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'", "https://cdn.example.com"],
    styleSrc: ["'self'", "'unsafe-inline'"],
    imgSrc: ["'self'", "https://images.example.com", "data:"],
    connectSrc: ["'self'", "https://api.example.com"],
    fontSrc: ["'self'", "https://fonts.googleapis.com"],
    objectSrc: ["'none'"],
    upgradeInsecureRequests: [],
  },
}));
```

7. Proteções Específicas

SQL Injection

```
[typescript]
// Nunca fazer
const users = await prisma.$queryRawUnsafe(`SELECT * FROM users`);

// Prisma previne com query builders
const user = await prisma.user.findUnique({ where: { email } });

// Se precisar de raw query, usar parametrização
const users = await prisma.$queryRaw`SELECT * FROM users WHERE`;
```

XSS (Cross-Site Scripting)

```
[typescript]
// Renderizar HTML sem sanitizar
<div dangerouslySetInnerHTML={{ __html: userComment }} />

// Sanitizar antes de renderizar
import DOMPurify from 'isomorphic-dompurify';
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(userComment)}} />

// No backend, escapar output
const sanitized = escapeHtml(userComment);
```

CSRF (Cross-Site Request Forgery)

```
[typescript]
// NestJS com CSRF protection
import * as csrf from 'csrf';

app.use(csrf({ cookie: true }));

// Enviar token CSRF em formulários/headers
// <meta name="csrf-token" content="{{csrfToken}}">
// Header: X-CSRF-Token
```


8. Secrets Management

O que NÃO fazer

```
[typescript]
// Hardcoded no código
const API_KEY = 'sk-1234567890abcdef';
const DB_PASSWORD = 'minha-senha-super-secreta';

// No .env comitado
.env
DB_PASSWORD=minha-senha

// No código fonte
config.service.apiKey = 'sk-1234567890abcdef';
```

O que fazer

```
[bash]
# .env.example (commitado, sem valores reais)
DATABASE_URL=postgresql://user:password@localhost:5432/db
JWT_SECRET=change-me
API_KEY=change-me

# .env (NÃO commitado, .gitignore)
DATABASE_URL=postgresql://user:senha-real@produção:5432/db
JWT_SECRET=minha-chave-jwt-segura
API_KEY=sk-real-key

# Validação na inicialização
# main.ts
const requiredEnvVars = ['DATABASE_URL', 'JWT_SECRET', 'API_KEY'];
for (const varName of requiredEnvVars) {
  if (!process.env[varName]) {
    throw new Error(`Variável de ambiente ${varName} não configurada`);
  }
}
```

9. Auditoria de Segurança

O que auditar

```
[typescript]
// Log de ações sensíveis
@Injectable()
export class AuditService {
  async log(action: AuditAction): Promise<void> {
    await prisma.auditLog.create({
      data: {
        userId: action.userId,
        action: action.type,
        resource: action.resource,
        resourceId: action.resourceId,
        details: action.details,
        ip: action.ip,
        userAgent: action.userAgent,
        timestamp: new Date(),
      },
    });
  }
}

// Uso
@Post('/transfer')
async transfer(@Body() dto: TransferDto, @Req() req) {
  const result = await this.transferService.execute(dto);
  await this.auditService.log({
    userId: req.user.id,
    type: 'TRANSFER',
    resource: 'Account',
    resourceId: dto.fromAccountId,
    details: { to: dto.toAccountId, amount: dto.amount },
    ip: req.ip,
    userAgent: req.headers['user-agent'],
  });
  return result;
}
```

Checklist de segurança para code review

Segurança em code review:

- [] Todos os inputs são validados?
- [] Autenticação em todas as rotas protegidas?
- [] Autorização verifica propriedade do recurso?
- [] Senhas com hash (bcrypt, não MD5/SHA1)?
- [] Sem segredos no código?
- [] Rate limiting nos endpoints críticos?
- [] CORS configurado (não * em produção)?
- [] SQL injection prevenido (ORM)?
- [] Headers de segurança configurados?
- [] Logs não expõem dados sensíveis?

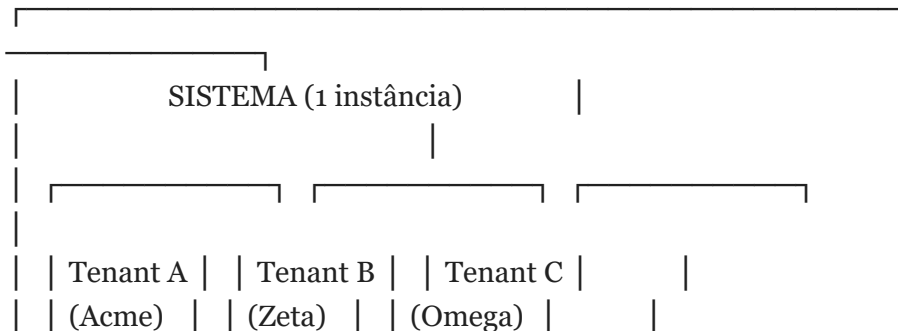
Arquitetura Multi-Tenant

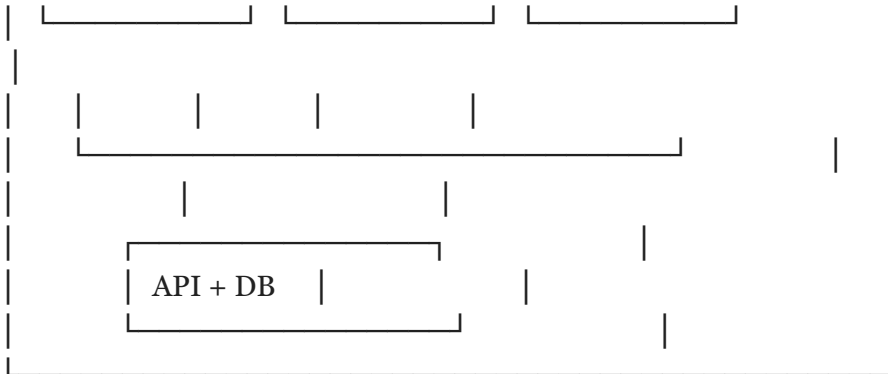
Módulo 13 -- Multi-Tenant: Construindo SaaS Escalável

****Como isolar dados, identidade e recursos entre clientes em um**

1. O que é Multi-Tenancy?

Multi-tenancy é um padrão arquitetural onde ****uma única instância**





1.1 Conceito

Diferente de **single-tenancy** (uma instância por cliente), no

Característica	Single-Tenant	Multi-Tenant
Instâncias	1 por cliente	1 para todos
Código	N versões	1 versão
Banco	N bancos	1 ou N (controlado)
Manutenção	N vezes	1 vez
Isolamento	Físico	Lógico / Físico

1.2 Por que usar?

Motivo	Descrição
Custo	Compartilhar infraestrutura reduz drasticamente o custo de desenvolvimento e manutenção.
Manutenção	Uma única base de código para atualizar, corrigir bugs e implementar novas funcionalidades.
Escalabilidade	Adicionar novos tenants sem provisionar servidores adicionais.
Time-to-market	Novo cliente entra em minutos, não semanas.
Atualizações	Todos os clientes recebem atualização simultaneamente.

1.3 Exemplos reais (SaaS)

- ```
- **Slack** -- cada workspace é um tenant; canais, mensagens e
- **Shopify** -- cada loja é um tenant; produtos, pedidos e cli
- **Notion** -- workspaces da equipe com páginas e permissões i
```

- **GitHub** -- organizações e repositórios como unidades de is

- **Salesforce** -- o pioneer do multi-tenancy enterprise; cada



### ### 1.4 Quando NÃO usar?





- Clientes exigem compliance físico (dados não podem co-habitar

- Poucos clientes com volumes massivos de dados

- Clientes exigem versões diferentes do software

- O custo de implementar isolamento lógico supera o de rodar N



---





## ## 2. Abordagens de Isolamento



Existem três estratégias principais para isolar dados entre ten



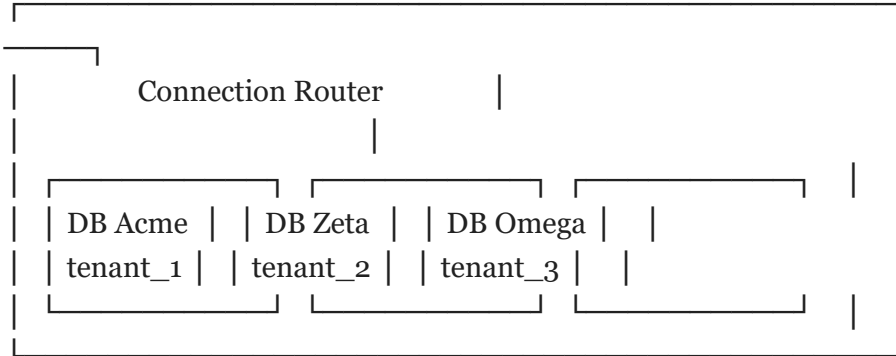
### ### 2.1 Database per Tenant



Cada tenant tem **\*\*seu próprio banco de dados\*\***. O roteador de c







```
// Router de conexão com lazy initialization
import { Pool } from 'pg';
const tenantDatabases: Record<string, string> = {
 acme: 'postgresql://user:pass@host:5432/acme_db',
 zeta: 'postgresql://user:pass@host:5432/zeta_db',
 omega: 'postgresql://user:pass@host:5432/omega_db',
};
class DatabaseRouter {
 private pools = new Map<string, Pool>();
 getPool(tenantId: string): Pool {
 if (!this.pools.has(tenantId)) {
 const url = tenantDatabases[tenantId];
 if (!url) throw new Error(Database not found for tenant:
${tenantId});
 this.pools.set(
 tenantId,
 new Pool({
 connectionString: url,
 max: 10,
 idleTimeoutMillis: 30000,
 })
);
 }
 }
}
```

```
return this.pools.get(tenantId)!;
```

```
}
```

```
} [text]
```

```
Vantagens:
```

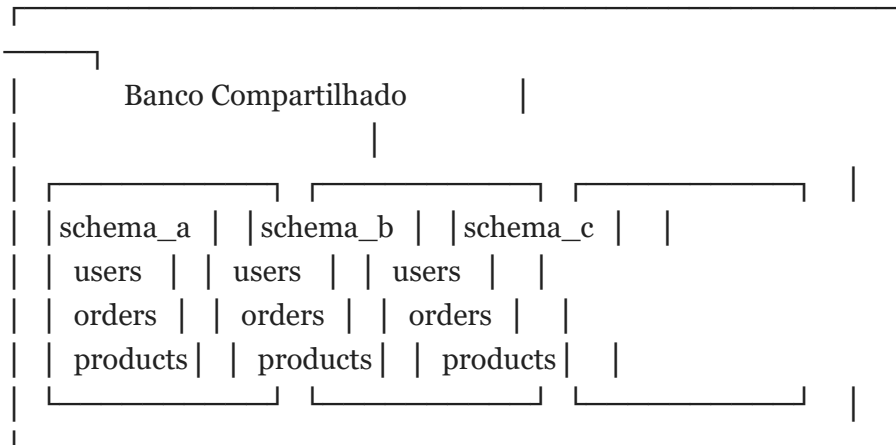
- **\*\*Isolamento total\*\*** -- um tenant nunca vê dados do outro, me
- **\*\*Backup/restore independente\*\*** -- pode restaurar um único te
- **\*\*SLAs diferenciados\*\*** -- cada banco pode estar em máquinas d
- **\*\*Migração para instância dedicada\*\*** -- basta apontar o DNS p
- **\*\*Compliance\*\*** -- atende requisitos LGPD/HIPAA que exigem sep

```
Desvantagens:
```

- **\*\*Custo\*\*** -- N bancos × licenciamento + infra; 1000 tenants =
- **\*\*Conexões\*\*** -- N tenants × pool size pode estourar limites d
- **\*\*Migrations\*\*** -- precisam rodar em N bancos; falha em um ten
- **\*\*Monitoramento\*\*** -- N bancos para monitorar, N alarmes para
- **\*\*Complexidade operacional\*\*** -- gerenciar N conexões, N backu

```
2.2 Schema per Tenant
```

Um banco de dados compartilhado, mas cada tenant tem seu **\*\*próprio**



```
-- Criar schema para novo tenant
CREATE SCHEMA IF NOT EXISTS tenant_acme;
-- Tabelas dentro do schema específico
CREATE TABLE tenant_acme.users (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 name TEXT NOT NULL,
 email TEXT UNIQUE NOT NULL,
 role TEXT NOT NULL DEFAULT 'member',
 created_at TIMESTAMPTZ DEFAULT NOW()
)
CREATE TABLE tenant_acme.orders (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 customer_id TEXT NOT NULL,
 total NUMERIC(10,2) NOT NULL,
 status TEXT NOT NULL DEFAULT 'pending',
 created_at TIMESTAMPTZ DEFAULT NOW()
)
```

↳ Query com schema dinâmico

```
SELECT * FROM tenant_acme.users WHERE email =
```

```
'[REDACTED]'
```

// Abstração para schema dinâmico

```
class SchemaTenantService {
 constructor(private tenantId: string) {}
 getSchemaName(): string {
 return tenant_`${this.tenantId}`;
 }
 async query<T = any>(sql: string, params?: any[]): Promise<T[]> {
 const schema = this.getSchemaName();
 // Substitui __SCHEMA__ pelo nome real do schema
 const finalSql = sql.replace(/__SCHEMA__/g, `${schema}`);
 const result = await pool.query(finalSql, params);
 return result.rows;
 }
 async createTenantSchema(): Promise<void> {
 const schema = this.getSchemaName();
```

```

// await pool.query(CREATE_SCHEMA_IF_NOT_EXISTS "${schema}");
// Rodar migrations para este schema
for (const migration of MIGRATIONS) {
 const sql = migration.sql.replace(/__SCHEMA__/g, "${schema}");
};
 await pool.query(sql);
}
}

```

[text]

#### **\*\*Vantagens:\*\***

- **\*\*Banco único\*\*** -- uma conexão, pool compartilhado, menos overhead
- **\*\*Backup centralizado\*\*** -- um dump = todos os tenants
- **\*\*Custo menor\*\*** -- uma licença de banco, uma máquina
- **\*\*Migrations mais simples\*\*** -- roda uma vez, altera múltiplos tenants
- **\*\*Transações entre schemas\*\*** -- possível (depende do banco)

#### **\*\*Desvantagens:\*\***

- **\*\*Isolamento por schema\*\*** -- não tão forte quanto DB separado
- **\*\*Ruídos between tenants\*\*** -- queries pesadas de um tenant com o outro
- **\*\*Recuperação de desastre\*\*** -- restaurar um tenant específico
- **\*\*Limite de schemas\*\*** -- PostgreSQL suporta milhares, mas MySQL não
- **\*\*Conexão única\*\*** -- um bottleneck na aplicação se todos os tenants

### ### 2.3 Shared Database (discriminador)

Um banco, um schema, e uma coluna `tenant\_id` em cada tabela com

```
// Modelo único com tenant_id
```

```

interface User {
 id: string;
 tenant_id: string; // discriminador obrigatório
 name: string;
 email: string;
 role: 'admin' | 'member';
 created_at: Date;
}

```

```

}
interface Order {
 id: string;
 tenant_id: string; // discriminador obrigatório
 customer_name: string;
 total: number;
 status: 'pending' | 'paid' | 'cancelled';
 created_at: Date;
}
// Repositório que SEMPRE filtra por tenant
class UserRepository {
 constructor(private tenantId: string) {}
 async findAll(): Promise<User[]> {
 return pool.query<User>(
 'SELECT * FROM users WHERE tenant_id = $1 ORDER BY
name',
 [this.tenantId]
).then(r => r.rows);
 }
 async findById(id: string): Promise<User | null> {
 const result = await pool.query<User>(
 'SELECT * FROM users WHERE id = $1 AND tenant_id = $2',
 [id, this.tenantId]
);
 return result.rows[0] || null;
 }
 async create(data: Omit<User, 'id' | 'tenant_id' | 'created_at'>):
Promise<User> {
 const result = await pool.query<User>(
 `INSERT INTO users (tenant_id, name, email, role)
VALUES ($1, $2, $3, $4)
RETURNING *`,
 [this.tenantId, data.name, data.email, data.role]
);
 }
}

```

```
);
return result.rows[0];
}
} [text]

Vantagens:
- **Máximo compartilhamento** -- um banco, uma conexão, custo n
- **Operação simples** -- add tenant = INSERT em tabela `tenant
- **Performance previsível** -- um pool, um plano de execução
- **Migrations tradicionais** -- funcionam como em app single-t
- **Baixa latência** -- sem roteamento de conexão

Desvantagens:
- **Menor isolamento** -- um `SELECT` sem `WHERE tenant_id = ?`
- **SQL injection** -- se um invasor consegue injetar SQL, cons
- **Índices inchados** -- a tabela cresce com todos os tenants,
- **Backup/restore global** -- não é possível restaurar um único
- **Difícil diferenciar SLAs** -- todos competem pelos mesmos
```

### ### 2.4 Comparação Detalhada

| Critério                           | DB per Tenant | Schema per Tenant | Shared DB      |
|------------------------------------|---------------|-------------------|----------------|
| ----- :----- :----- :-----         |               |                   |                |
| <b>**Isolamento**</b>              | Físico        | Lógico (schema)   | Lógico (linha) |
| <b>**Segurança**</b>               |               |                   |                |
| <b>**Performance**</b>             | (dedicado)    | (compartilhado)   | (índices       |
| <b>**Custo**</b>                   | (caro)        | (médio)           | (barato)       |
| <b>**Complexidade**</b>            | (média)       | (alta)            | (baixa)        |
| <b>**Migrations**</b>              | (N execuções) | (N schemas)       | (1 execução    |
| <b>**Backup/Restore**</b>          | (individual)  | (global)          | (só global)    |
| <b>**Escalabilidade**</b>          |               |                   |                |
| <b>**Adicionar tenant**</b>        | (criar DB)    | (criar schema)    | (INSE          |
| <b>**Customização por tenant**</b> |               |                   |                |

---

## ## 3. Análise Aprofundada por Dimensão

### ### 3.1 Segurança



**\*\*Database per Tenant:\*\*** Um vazamento de dados de um tenant não





**\*\*Schema per Tenant:\*\*** Um invasor que ganha acesso ao banco po



**\*\*Shared Database:\*\*** Um SQL injection ou bug no `WHERE` expõe \*

- Testes de isolamento obrigatórios (ver seção 13)

- Code review com checklist específico

- Row-Level Security (RLS) do PostgreSQL como camada extra





-- Row-Level Security no PostgreSQL

```
CREATE TABLE users (
 id UUID PRIMARY KEY,
 tenant_id TEXT NOT NULL,
 name TEXT NOT NULL,
 email TEXT NOT NULL
```

); Habilitar RLS

```
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
```

-- Política: usuário só vê firmas do seu tenant

```
CREATE POLICY tenant_isolation ON users
```

```
USING (tenant_id = current_setting('app.tenant_id')::TEXT);
```

-- Na aplicação, antes de qualquer query:

8

### ### 3.2 Performance

**\*\*Database per Tenant:\*\*** Cada tenant tem pool isolado. Um tenar

**\*\*Schema per Tenant:\*\*** Pool compartilhado. Uma query `SELECT \*

- Statement timeout (`SET statement\_timeout = '30s'`)
- `pg\_terminate\_backend` para queries runaway
- Resource queues (Citus, Yugabyte)

**\*\*Shared Database:\*\*** Todos competem pelo mesmo pool e índices.

- Rate limiting por tenant
- `pg\_stat\_statements` para monitorar queries pesadas
- Índices parciais por tenant (se houver padrões distintos)

-- Índice parcial para tenant com padrão específico

```
CREATE INDEX idx_orders_acme_large
ON orders (total DESC)
```

[text]

### ### 3.3 Custo

| Cenário | DB per Tenant | Schema per Tenant | Shared DB |
|---------|---------------|-------------------|-----------|
| -----   | :-----:       | :-----:           | :-----:   |

| 10 tenants | 10× RDS db.t3.medium | 1× RDS db.r5.large | 1× F

| Custo/mês | ~\$150 | ~\$50 | ~\$50 |

| 100 tenants | 100× tiny | 1× db.r5.xlarge | 1× db.r5.xlarge |

| Custo/mês | ~\$500 | ~\$200 | ~\$200 |

| 1000 tenants | Inviável | 1x db.r5.2xlarge + sharding | 1x db

| Custo/mês | -- | ~\$700 | ~\$1400 |





**\*\*Database per Tenant\*\*** não escala financeiramente além de algu



### ### 3.4 Complexidade Operacional



| Operação | DB per Tenant | Schema per Tenant | Shared DB |

|-----| :-----: | :-----: | :-----: |

| Adicionar tenant | Criar DB + rodar migrations | Criar schema

| Backup | N backups (1 por DB) | 1 backup do banco | 1 backup



| Restore individual | Restaurar DB específico | Exportar schema

| Restore total | N restores | 1 restore | 1 restore |

| Upgrade de schema | Roda em N bancos | 1 script para N schema

| Rollback | Rollback em N bancos | 1 rollback para N schemas |

| Migração de plano | Sobe de instância | Altera resource limit



---





## ## 4. Identificação do Tenant



O sistema precisa identificar \*\*qual tenant está fazendo a requ



### ### 4.1 Estratégias de Identificação



| Estratégia | Exemplo | Segurança | Complexidade | Ideal para

|-----|-----|:-----:|:-----:|-----|



| **\*\*Subdomínio\*\*** | `acme.minhasaas.com` | Média | Média | Apps

| **\*\*Header HTTP\*\*** | ``X-Tenant-Id: acme`` | [Média](#) | [Baixa](#) | [APIs](#)

| **\*\*JWT Claim\*\*** | `{ "tid": "acme" }` | Alta | Média | Apps aut

| **\*\*Path Parameter\*\*** | `/api/acme/users`` | Baixa | Baixa | Desse

| \*\*DNS + SSL\*\* | SNI com cert por tenant | Alta | Alta | Enter



### ### 4.2 Subdomínio





```
// tenant.extractor.ts
function extractTenantFromHost(host: string): string | null {
 // host: "acme.minhasaas.com" -> "acme"
 // host: "localhost:3000" -> "localhost" (não é tenant)
 const parts = host.split('.');
 if (parts.length < 3) return null; // sem subdomínio
 return parts[0];
}
// Em produção, validar contra lista de tenants permitidos
async function validateTenantSubdomain(
 host: string,
 tenantService: TenantService
): Promise<Tenant | null> {
 const subdomain = extractTenantFromHost(host);
 if (!subdomain) return null;
 return tenantService.findBySubdomain(subdomain);
}
```

### ### 4.3 Header HTTP

// Extração simples e direta

```
import { Request } from 'express';
const TENANT_HEADER = 'tenant-id';
function extractTenantFromHeader(req: Request): string | undefined {
 {
 const tenantId = req.headers[TENANT_HEADER];
 if (Array.isArray(tenantId)) return tenantId[0];
 return tenantId;
 }
}
// Com validação de formato (slug)
const TENANT_SLUG_REGEX = /^[a-z0-9-]{3,50}$/;
function validateTenantSlug(slug: string): boolean {
 return TENANT_SLUG_REGEX.test(slug);
}
```

[text]

### ### 4.4 JWT Claim



```
// Interface do payload
interface TenantJwtPayload {
 sub: string; // user ID
 tid: string; // tenant ID
 rol: string; // role dentro do tenant
 exp: number; // expiração
}
// Extrair tenant do JWT
import * as jwt from 'jsonwebtoken';
function extractTenantFromJwt(authHeader?: string): string | null {
 if (!authHeader) return null;
 const token = authHeader.replace('Bearer ', '');
 try {
 const payload = jwt.verify(
 token,
 process.env.JWT_SECRET!
) as TenantJwtPayload;
 return payload.tid || null;
 } catch {
 return null; // token inválido
 }
}
```

### ### 4.5 Path Parameter

```
// Útil para debug, mas não recomendado para produção
// Exemplo: GET /api/:tenant/users
@Get('/api/:tenant/users')
findUsers(@Param('tenant') tenantId: string) {
 return this.userService.findAll(tenantId);
}
```

### ### 4.6 Estratégia Combinada (Fallback)

```
function resolveTenant(req: Request): string {
```

```
// Prioridade: JWT -> Header -> Subdomínio -> Path
return (
 extractTenantFromJwt(req.headers.authorization) ??
 extractTenantFromHeader(req) ??
 extractTenantFromHost(req.hostname) ??
 extractTenantFromPath(req.path) ??
 () => { throw new BadRequestException('Tenant não
identificado'); })()
);
}
```

```

```

```
5. Middleware de Tenant
```

```
5.1 Implementação com NestJS
```

```
// tenant.middleware.ts
import { Injectable, NestMiddleware, UnauthorizedException } from
 '@nestjs/common';
import { Request, Response, NextFunction } from 'express';
import * as jwt from 'jsonwebtoken';
interface TenantInfo {
 id: string;
 name: string;
 plan: 'free' | 'pro' | 'enterprise';
 active: boolean;
}
@Injectable()
export class TenantMiddleware implements NestMiddleware {
 constructor() {
 // Em produção, carregar de um cache (Redis) ou banco
 this.loadTenants();
 }
 private tenants = new Map<string, TenantInfo>();
 private loadTenants() {
```

```

this.tenants.set('acme', {
 id: 'acme', name: 'Acme Inc.', plan: 'enterprise', active: true,
});
this.tenants.set('zeta', {
 id: 'zeta', name: 'Zeta Ltda.', plan: 'pro', active: true,
});
this.tenants.set('omega', {
 id: 'omega', name: 'Omega S.A.', plan: 'free', active: true,
});
use(req: Request, res: Response, next: NextFunction) {
 const tenantId = this.extractTenantId(req);
 const tenant = this.tenants.get(tenantId);
 if (!tenant) {
 throw new UnauthorizedException(Tenant "${tenantId}" não
 encontrado);
 }
 if (!tenant.active) {
 throw new UnauthorizedException(Tenant "${tenantId}" está
 inativo);
 }
 // Anexa ao request para uso nos controllers
 (req as any).tenant = tenant;
 (req as any).tenantId = tenant.id;
 next();
}
private extractTenantId(req: Request): string {
 const fromJwt = this.extractFromJwt(req.headers.authorization);
 if (fromJwt) return fromJwt;
 const fromHeader = req.headers['x-tenant-id'] as string;
 if (fromHeader) return fromHeader;
 const fromSubdomain = req.hostname.split('.')[0];
 if (fromSubdomain && fromSubdomain !== 'www' &&
 fromSubdomain !== 'app') {
 return fromSubdomain;
 }
 throw new UnauthorizedException(
 'Identificação do tenant é obrigatória (JWT, X-Tenant-Id ou

```

```

subdomínio)'
);
private extractFromJwt(authorization?: string): string | null {
 if (!authorization) return null;
 try {
 const token = authorization.replace('Bearer ', '');
 const payload = jwt.verify(token, process.env.JWT_SECRET!) as
any;
 return payload.tid || null;
 } catch {
 return null;
 }
}
} [text]

```

### ### 5.2 Aplicação Global ou por Rota

```

// app.module.ts
import { Module, NestModule, MiddlewareConsumer } from
 '@nestjs/common';
import { TenantMiddleware } from './tenant/tenant.middleware';
@Module({
 imports: [TenantModule],
})
export class AppModule implements NestModule {
 configure(consumer: MiddlewareConsumer) {
 consumer
 .apply(TenantMiddleware)
 .exclude(
 'auth/(.*)', // login não precisa de tenant
 'health', // health check
 'public/(.*)', // páginas públicas

```

```

)
 .forRoutes('*');
 }
}

```

### ### 5.3 TenantModule

```

// tenant.module.ts
import { Global, Module } from '@nestjs/common';
import { TenantService } from './tenant.service';
import { TenantMiddleware } from './tenant.middleware';
@Global() // disponível em toda a aplicação
@Module({
 providers: [TenantService, TenantMiddleware],
 exports: [TenantService],
})

```

```

6 [text]

```

### ### 5.4 TenantService

```

// tenant.service.ts
import { Injectable, Scope } from '@nestjs/common';
interface Tenant {
 id: string;
 name: string;
 plan: 'free' | 'pro' | 'enterprise';
}
@Injectable({ scope: Scope.REQUEST })
export class TenantService {
 private tenant!: Tenant;
 setTenant(tenant: Tenant) {
 this.tenant = tenant;
 }
 getTenant(): Tenant {
 if (!this.tenant) {
 throw new Error('Tenant não configurado para esta requisição');
 }
 }
}

```

```

 }
 return this.tenant;
 }
 getTenantId(): string {
 return this.getTenant().id;
 }
 getPlan(): string {
 return this.getTenant().plan;
 }
 isEnterprise(): boolean {
 return this.getPlan() === 'enterprise';
 }
}

```

### ### 5.5 @Tenant() Decorator

```

// tenant.decorator.ts
import { createParamDecorator, ExecutionContext } from
'@nestjs/common';
export const Tenant = createParamDecorator(
 (data: keyof Tenant | undefined, ctx: ExecutionContext) => {
 const request = ctx.switchToHttp().getRequest();
 const tenant = request.tenant;
 if (!tenant) {
 throw new Error("Tenant não encontrado no request.
TenantMiddleware está configurado?");
 }
 return data ? tenant[data] : tenant;
 }
);

// Uso no controller
@Get('users')
findAll(@Tenant() tenant: Tenant) {
 return this.userService.findAll(tenant.id);
}
@Get('plan')
getPlan(@Tenant('plan') plan: string) {
 return { plan };
}

```



```
} [text]
```

### ### 5.6 AsyncLocalStorage para Contexto

```
// tenant-context.ts
import { AsyncLocalStorage } from 'async_hooks';
export interface TenantContext {
 tenantId: string;
 tenantName: string;
 plan: string;
}
export const tenantContext = new
AsyncLocalStorage<TenantContext>();
// tenant-context.middleware.ts
import { Injectable, NestMiddleware } from '@nestjs/common';
@Injectable()
export class TenantContextMiddleware implements NestMiddleware {
 use(req: any, res: any, next: () => void) {
 const context: TenantContext = {
 tenantId: req.tenantId,
 tenantName: req.tenant?.name,
 plan: req.tenant?.plan,
 };
 tenantContext.run(context, () => next());
 }
}
// Uso em qualquer parte do código
import { tenantContext } from './tenant-context';
function getCurrentTenantId(): string {
 const ctx = tenantContext.getStore();
 if (!ctx) throw new Error('Fora de contexto de tenant');
 return ctx.tenantId;
}

```

### ## 6. Prisma Multi-Tenant

### ### 6.1 Schema per Tenant com Prisma



```
// schema.prisma -- modelo base
generator client {
 provider = "prisma-client-js"
}
datasource db {
 provider = "postgresql"
 url = env("DATABASE_URL")
}
// Modelo compartilhado (global)
model Tenant {
 id String @id @default(uuid())
 slug String @unique
 name String
 plan String @default("free")
 active Boolean @default(true)
 createdAt DateTime @default(now()) @map("created_at")
 @@map("tenants")
}
// Modelos por tenant (Prisma não suporta schema dinâmico
nativamente)
// Solução: Client per tenant
model User {
 id String @id @default(uuid())
 name String
 email String @unique
 role String @default("member")
 createdAt DateTime @default(now()) @map("created_at")
 @@map("users")
}
model Project {
 id String @id @default(uuid())
 name String
 createdAt DateTime @default(now()) @map("created_at")
 @@map("projects")
}
```

```

} [typescript]

// prisma-multi-tenant.ts
import { PrismaClient } from '@prisma/client';
class PrismaTenantManager {
 private clients = new Map<string, PrismaClient>();
 getClient(schema: string): PrismaClient {
 if (!this.clients.has(schema)) {
 const url = new URL(process.env.DATABASE_URL!);
 url.searchParams.set('schema', schema);
 const client = new PrismaClient({
 datasources: {
 db: { url: url.toString() },
 },
 });
 this.clients.set(schema, client);
 }
 return this.clients.get(schema)!;
 }
 async closeAll(): Promise<void> {
 for (const [schema, client] of this.clients) {
 await client.$disconnect();
 }
 }
}

```

### ### 6.2 Shared Database com Prisma

```

// schema.prisma -- shared database
model User {
 id String @id @default(uuid())
 tenantId String @map("tenant_id")
 name String
 email String
 role String @default("member")
}

```

```

 createdAt DateTime @default(now()) @map("created_at")
 @@index([tenantId])
 @@index([tenantId, email])
 @@map("users")
 }
 model Order {
 id String @id @default(uuid())
 tenantId String @map("tenant_id")
 total Decimal @db.Decimal(10, 2)
 status String @default("pending")
 createdAt DateTime @default(now()) @map("created_at")
 @@index([tenantId, createdAt])
 @@map("orders")
 }
} [typescript]

```

```

// tenant-aware.service.ts
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';
import { TenantService } from './tenant.service';
@Injectable()
export class TenantAwareService {
 constructor(
 private prisma: PrismaService,
 private tenantService: TenantService,
) {}
 private get tenantId() {
 return this.tenantService.getTenantId();
 }
 async findUsers() {
 return this.prisma.user.findMany({
 where: { tenantId: this.tenantId },
 });
 }
 async createUser(data: { name: string; email: string; role?: string }) {
 return this.prisma.user.create({
 data: {

```

```

 ...data,
 tenantId: this.tenantId,
 },
});

async findOrdersByDateRange(start: Date, end: Date) {
 return this.prisma.order.findMany({
 where: {
 tenantId: this.tenantId,
 createdAt: { gte: start, lte: end },
 },
 orderBy: { createdAt: 'desc' },
 });
}
}
}

```

### ### 6.3 Prisma Middleware para Tenant

```

// prisma-tenant.middleware.ts
import { PrismaClient } from '@prisma/client';
import { tenantContext } from './tenant-context';
export function createTenantMiddleware(prisma: PrismaClient): void {
 {
 prisma.$use(async (params, next) => {
 const ctx = tenantContext.getStore();
 if (!ctx) return next(params);
 // Adiciona tenantId automaticamente em creates
 if (params.action === 'create' && params.args.data) {
 params.args.data.tenantId = ctx.tenantId;
 }
 // Adiciona filtro de tenant em finds
 if (
 ['findMany', 'findFirst', 'findUnique', 'update',
 'delete'].includes(params.action)
) {

```

```

 const where = params.args.where ?? {};
 where.tenantId = ctx.tenantId;
 params.args.where = where;
 }
 return next(params);
});

```

```
// Inicialização
```

```
const prisma = new PrismaClient();
```

```

[
 [text]
]

```

```
6.4 Prisma Extension (Prisma >= 5.0)
```

```
// tenant.extension.ts
```

```
import { PrismaClient } from '@prisma/client';
```

```
import { tenantContext } from './tenant-context';
```

```
export const tenantExtension = Prisma.defineExtension((client) => {
```

```
 return client.$extends({
```

```
 query: {
```

```
 $allModels: {
```

```
 async $allOperations({ model, operation, args, query }) {
```

```
 const ctx = tenantContext.getStore();
```

```
 if (ctx && ['create', 'createMany'].includes(operation)) {
```

```
 args.data = { ...args.data, tenantId: ctx.tenantId };
```

```
 if (ctx && ['findMany', 'findFirst', 'update',
```

```
 'delete'].includes(operation)) {
```

```
 args.where = { ...args.where, tenantId: ctx.tenantId };
```

```
 return query(args);
```

```
 },
```

```
 },
```

```
 },
```

```
 });
```

```
});
```

```
// Uso
```



```

7. Migrations Multi-Tenant

7.1 Database per Tenant

// migrate-all-tenants.ts
import { Pool } from 'pg';
import { readMigrationFiles } from './migration-runner';
async function migrateAllTenants(): Promise<void> {
 const tenants = [
 { id: 'acme', dbUrl: 'postgresql://.../acme' },
 { id: 'zeta', dbUrl: 'postgresql://.../zeta' },
 { id: 'omega', dbUrl: 'postgresql://.../omega' },
];
 const migrations = await readMigrationFiles();
 for (const tenant of tenants) {
 console.log([`${tenant.id}`] Iniciando migrations...);
 const pool = new Pool({ connectionString: tenant.dbUrl });
 try {
 for (const migration of migrations) {
 await pool.query('BEGIN');
 try {
 await pool.query(migration.sql);
 await pool.query(
 INSERT INTO _migrations (name, applied_at) VALUES
($1, NOW()),
 [migration.name]
);
 await pool.query('COMMIT');
 console.log([`${tenant.id}`] ${migration.name});
 } catch (err) {
 await pool.query('ROLLBACK');
 console.error([`${tenant.id}`] ${migration.name}: , err);
 }
 }
 } catch (err) {
 console.error([`${tenant.id}`] ${migration.name}: , err);
 }
 }
}

```

```

 throw err; // abortar todos? ou continuar?
 }
}
} finally {
 await pool.end();
}
}
}
} [text]

```

### ### 7.2 Schema per Tenant

```

// schema-migration-runner.ts
const pool = new Pool({ connectionString:
process.env.DATABASE_URL });
const MIGRATIONS = [
{
 name: '001_create_users',
 sql: `
CREATE TABLE IF NOT EXISTS __SCHEMA__.users (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 name TEXT NOT NULL,
 email TEXT NOT NULL,
 role TEXT NOT NULL DEFAULT 'member',
 created_at TIMESTAMPTZ DEFAULT NOW()
);
`,
},
{
 name: '002_add_phone',
 sql: `
ALTER TABLE __SCHEMA__.users
ADD COLUMN IF NOT EXISTS phone TEXT;

```

```

 `
 },
 {
 name: '003_create_projects',
 sql: `
 CREATE TABLE IF NOT EXISTS __SCHEMA__.projects (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 name TEXT NOT NULL,
 description TEXT,
 created_at TIMESTAMPTZ DEFAULT NOW()
);
 `,
 },
];
async function migrateSchema(schema: string): Promise<void> {
 const client = await pool.connect();
 try {
 // Criar schema se não existe
 await client.query(CREATE_SCHEMA_IF_NOT_EXISTS`${schema}`);
 }
 // Criar tabela de controle de migrations no schema
 await client.query(`
 CREATE TABLE IF NOT EXISTS "${schema}."_migrations (
 name TEXT PRIMARY KEY,
 applied_at TIMESTAMPTZ DEFAULT NOW()
)
 `);
 for (const migration of MIGRATIONS) {
 // Verificar se já foi aplicada
 const exists = await client.query(
 SELECT_1_FROM`${schema}."_migrations WHERE name =
 $1,
 [migration.name]
);
 if (exists.rows.length > 0) continue;
}

```

```
// Aplicar migration
const sql = migration.sql.replace(/__SCHEMA__/g, `${schema}`);
);
await client.query('BEGIN');
try {
 await client.query(sql);
 await client.query(
 INSERT INTO `${schema}`._migrations (name) VALUES
($1),
 [migration.name]
);
 await client.query('COMMIT');
 console.log([`${schema}`] `${migration.name}`);
} catch (err) {
 await client.query('ROLLBACK');
 throw err;
}
}
} finally {
 client.release();
}
}
} async function migrateAllSchemas(): Promise<void> {
 const tenants = await pool.query('SELECT slug FROM tenants
WHERE active = true');
 for (const tenant of tenants.rows) {
 await migrateSchema(tenant_`${tenant.slug}`);
 }
}
```

### ### 7.3 Estratégias de Rollback

```
// migrate-with-transaction.ts
```

```

async function migrateTenantWithTransaction(
 tenantId: string,
 migrationSql: string
): Promise<void> {
 const schema = tenant_`${tenantId}`;
 const client = await pool.connect();
 try {
 await client.query('BEGIN');
 // Isolar a transação no schema do tenant
 await client.query(SET search_path TO `${schema}`);
 await client.query(INSERT INTO _migrations (name) VALUES
 ($1), [
 '004_add_status',
]);
 await client.query('COMMIT');
 console.log(`${tenantId}] Migration aplicada com sucesso`
);
 } catch (error) {
 await client.query('ROLLBACK');
 console.error(`${tenantId}] Migration falhou, rollback
 executado);
 throw error;
 } finally {
 client.release();
 }
}

```

### ### 7.4 Shared Database

```

// shared-migration.ts
// Migrations tradicionais -- funcionam como app single-tenant
async function migrateShared(): Promise<void> {
 const client = await pool.connect();
 try {

```

```

// await client.query('BEGIN');
// Adicionar coluna tenant_id se não existe
await client.query(`
 ALTER TABLE users
 ADD COLUMN IF NOT EXISTS tenant_id TEXT NOT NULL
 DEFAULT 'default'
`);
// Índices compostos para performance
await client.query(`
 CREATE INDEX IF NOT EXISTS idx_users_tenant_email
 ON users (tenant_id, email)
`);
await client.query(`
 CREATE INDEX IF NOT EXISTS idx_orders_tenant_created
 ON orders (tenant_id, created_at DESC)
`);
await client.query('COMMIT');
} catch (error) {
 await client.query('ROLLBACK');
 throw error;
} finally {
 client.release();
}
}

```

```

```

```
8. Dados Compartilhados vs Por Tenant
```

```
8.1 Tabelas Globais (Compartilhadas)
```

```
Dados que **não pertencem a nenhum tenant específico** e são co
```

```

// Globais -- uma única instância para o sistema todo
interface Plan {
 id: string;

```

```

name: string; // "Free", "Pro", "Enterprise"
maxUsers: number;
maxStorage: number; // MB
monthlyPrice: number;
features: string[]; // ["api_access", "custom_domain",
"advanced_reports"]
}
interface FeatureFlag {
 id: string;
 key: string; // "audit_log"
 description: string;
 enabledByDefault: boolean;
}
interface SystemConfig {
 key: string; // "max_upload_size_mb"
 value: string;
 description: string;
}
interface AuditLog {
 id: string;
 tenantId: string;
 action: string;
 userId: string;
 metadata: JSON;
 createdAt: Date;
}

```

### ### 8.2 Tabelas Por Tenant (Isoladas)

Dados que **\*\*pertencem a um tenant específico\*\*** e devem ser isol

// Isoladas por tenant

```

interface User {
 id: string;

```

```
tenantId: string; // FK implícita para o tenant
name: string;
email: string;
role: 'admin' | 'member' | 'viewer';
createdAt: Date;
}
interface Project {
 id: string;
 tenantId: string;
 name: string;
 description?: string;
 status: 'active' | 'archived';
 createdAt: Date;
}
interface Task {
 id: string;
 tenantId: string;
 projectId: string;
 title: string;
 assigneeId?: string;
 status: 'todo' | 'doing' | 'done';
 createdAt: Date;
}
interface Invoice {
 id: string;
 tenantId: string;
 number: string;
 amount: number;
 status: 'pending' | 'paid' | 'cancelled';
 dueDate: Date;
}
```

### ### 8.3 Regra Prática



| DADOS GLOBAIS (shared)     |                   |
|----------------------------|-------------------|
| • Planos e preços          | • Feature flags   |
| • Configurações do sistema | • Templates       |
| • Catálogo de integrações  | • Regiões/Países  |
| • Auditoria centralizada   | • Webhooks system |

| DADOS POR TENANT (isolados) |                       |
|-----------------------------|-----------------------|
| • Usuários e permissões     | • Projetos e tarefas  |
| • Faturas e pagamentos      | • Produtos e catálogo |
| • Configurações do tenant   | • Logs de atividade   |
| • Arquivos e uploads        | • Relatórios gerados  |

### ### 8.4 Implementação em Schema per Tenant

-- Schema global (público)

```
CREATE SCHEMA IF NOT EXISTS public;
CREATE TABLE public.plans (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 name TEXT NOT NULL,
 max_users INT NOT NULL,
 monthly_price NUMERIC(10,2) NOT NULL,
 features JSONB NOT NULL DEFAULT '[]'
);
CREATE TABLE public.tenants (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 slug TEXT UNIQUE NOT NULL,
```

```
name TEXT NOT NULL,
plan_id UUID REFERENCES public.plans(id),
active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
```

↳ Schema do tenant

```
CREATE SCHEMA IF NOT EXISTS tenant_acme;
CREATE TABLE tenant_acme.users (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID REFERENCES public.tenants(id),
 name TEXT NOT NULL,
 email TEXT NOT NULL,
 role TEXT NOT NULL DEFAULT 'member',
 created_at TIMESTAMPTZ DEFAULT NOW())
```

↳ Query que cruza dado global com dado do tenant

```
SELECT u.name, p.name as plan_name
FROM tenant_acme.users u
JOIN public.tenants t ON t.id = u.tenant_id
JOIN public.plans p ON p.id = t.plan_id
```

```
[text]

9. Seed por Tenant

9.1 Seed Automático ao Criar Tenant
```

```
// tenant-seed.ts
async function seedNewTenant(tenantSlug: string, plan: string):
Promise<void> {
 const schema = tenant_`${tenantSlug}`;
 try {
 const client = await pool.connect();
 await client.query(CREATE SCHEMA IF NOT EXISTS `${schema}`)
```

```
}/ 1. Migrations básicas
```

```
await client.query(`
 CREATE TABLE IF NOT EXISTS "${schema}".users (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 name TEXT NOT NULL,
 email TEXT NOT NULL,
 role TEXT NOT NULL DEFAULT 'member',
 created_at TIMESTAMPTZ DEFAULT NOW()
)
`)
```

```
await client.query(`
 CREATE TABLE IF NOT EXISTS "${schema}".projects (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 name TEXT NOT NULL,
 description TEXT,
 status TEXT DEFAULT 'active',
 created_at TIMESTAMPTZ DEFAULT NOW()
)
`)
```

```
await client.query(`
 CREATE TABLE IF NOT EXISTS "${schema}".settings (
 key TEXT PRIMARY KEY,
 value TEXT NOT NULL
)
`)
```

```
// 2: Seed data padrão
```

```
await client.query(`
 INSERT INTO "${schema}".users (name, email, role)
 VALUES ('Admin', 'admin@${tenantSlug}.com', 'admin')
`)
```

```
await client.query(`
 INSERT INTO "${schema}".settings (key, value) VALUES
 ('locale', 'pt-BR'),
 ('timezone', 'America/Sao_Paulo'),
`)
```

```

 ('notification_email', 'true'),
 ('max_upload_size_mb', '10'),
 ('default_language', 'pt')
 `);
// 3. Dados condicionais por plano
if (plan === 'enterprise') {
 await client.query(`
 INSERT INTO "${schema}".settings (key, value) VALUES
 ('custom_domain', ''),
 ('sso_enabled', 'true'),
 ('audit_log_retention_days', '365')
 `);
}
console.log([`${tenantSlug}] Seed concluído`);
} catch (error) {
 console.error([`${tenantSlug}] Seed falhou:`, error);
 throw error;
} finally {
 client.release();
}
}

// Hook na criação do tenant
async function createTenant(data: { slug: string; name: string; plan: string }) {
 const result = await pool.query(
 `INSERT INTO tenants (slug, name, plan_id)
 VALUES ($1, $2, (SELECT id FROM plans WHERE name = $3))
 RETURNING *`,
 [data.slug, data.name, data.plan]
);
 const tenant = result.rows[0];
 await seedNewTenant(tenant.slug, data.plan);
 return tenant;
}

```

### ### 9.2 Comandos CLI (NestJS Command)



```
// tenant.command.ts
import { Command, CommandRunner } from 'nest-commander';
import { TenantService } from './tenant.service';
@Command({ name: 'tenant:create', description: 'Cria novo tenant
com seed' })
export class TenantCommand extends CommandRunner {
 constructor(private tenantService: TenantService) {
 super();
 }
 async run(inputs: string[]): Promise<void> {
 const [slug, name, plan = 'free'] = inputs;
 if (!slug || !name) {
 console.error('Uso: tenant:create <slug> <name> [plan]');
 process.exit(1);
 }
 console.log(Criando tenant: ${slug} (${name}) - Plano:
 ${plan});
 const tenant = await this.tenantService.createTenant({ slug, name,
 plan });
 console.log(Tenant ${tenant.slug} criado com sucesso!);
 }
}
```

### ### 9.3 Idempotência

```
// Seeds devem ser idempotentes (podem rodar múltiplas vezes)
async function seedSettingsIfNotExists(schema: string):
Promise<void> {
 const defaultSettings = [
 { key: 'locale', value: 'pt-BR' },
 { key: 'timezone', value: 'America/Sao_Paulo' },
 { key: 'notification_email', value: 'true' },
];
 for (const setting of defaultSettings) {
 await pool.query(`
```

```
INSERT INTO "${schema}".settings (key, value)
VALUES ($1, $2)
ON CONFLICT (key) DO NOTHING
`, [setting.key, setting.value]);
}
}
```

```

```

```
10. Backup e Restore
```

```
10.1 Database per Tenant
```

```
#!/bin/bash
```

## backup-all-tenants.sh -- backup individual por tenant

---

```
TENANTS=("acme" "zeta" "omega")
DATE=$(date +%Y%m%d_%H%M)
BACKUP_DIR="/backups"
for TENANT in ${TENANTS[@]}; do
 echo "Iniciando backup do tenant: $TENANT"
 pg_dump postgresql://user:pass@localhost:5432/${TENANT}_db" \
 --format=custom \
 --compress=9 \
 file="${BACKUP_DIR}/${TENANT}_${DATE}.dump"
 if [$? -eq 0]; then
 echo " Backup de $TENANT concluído:
 ${BACKUP_DIR}/${TENANT}_${DATE}.dump"
 else
 echo " Falha no backup de $TENANT"
 fi
done
```



## Restore individual

---

*pg\_restore --dbname=postgresql://user:pass@localhost:5432/acme\_db \ | Architettura Backend Enterprise*

**pg\_restore**

**--dbname=postgresql://user:pass@localhost:5432/acme\_db \**

---

**--clean --if-exists \**

---

backups/acme\_20240101.dump

---

[text]

```
10.2 Schema per Tenant -- Backup Seletivo
```

```
#!/bin/bash
```

## backup-schema.sh -- backup de schema específico

---

```
TENANT=$1
DATE=$(date +%Y%m%d)
if [-z "$TENANT"]; then
 echo "Uso: $0 <tenant_slug>"
 exit 1
fi
DB_URL="postgresql://user:pass@localhost:5432/shared_db"
```

## Backup do schema específico (PostgreSQL 15+)

---

```
pg_dump "$DB_URL" \
--schema="tenant_${TENANT}" \
--format=custom \
--compress=9 \
--file="backups/schema_${TENANT}_${DATE}.dump" \
echo "Schema tenant_${TENANT} salvo em \
backups/schema_${TENANT}_${DATE}.dump"
```

## Restore

---

`pg_restore "$DB_URL" \`

---



`--schema="tenant_${TENANT}" \` | *Arquitetura Backend Enterprise*

`--schema="tenant_${TENANT}" \`

---

--clean \

---

## backups/schema\_zeta\_20240101.dump

## ### 10.3 Estratégia por Plano

| Plano          | Backup                           | RPO (Recovery Point Objective) | RTO (Recovery Time Objective) |
|----------------|----------------------------------|--------------------------------|-------------------------------|
| **Free**       | Diário compartilhado             | 24h   4h                       | 4h   1h                       |
| **Pro**        | Diário + WAL archiving (PITR)    | 1h   1h                        | 1h   1h                       |
| **Enterprise** | Backup dedicado + PITR + réplica | 5min   15min                   | 5min   15min                  |

```
// backup-scheduler.ts
```

```
interface BackupPolicy {
```

```
 type: 'shared' | 'dedicated' | 'pitr';
```

```
 frequency: string; // cron expression
```

```
 retentionDays: number;
```

```
}
const planBackupPolicies: Record<string, BackupPolicy> = {
 free: { type: 'shared', frequency: '0 2 *', retentionDays: 7 },
 pro: { type: 'pitr', frequency: '0 /6 ', retentionDays: 30 },
 enterprise: { type: 'dedicated', frequency: '0 /1 ', retentionDays: 90 },
},
```

```
async function scheduleTenantBackup(tenant: { id: string; plan: string }): Promise<void> {
```

```
 const policy = planBackupPolicies[tenant.plan];
 switch (policy.type) {
```

```
 case 'shared':
```

```
 // Backup global já cobre
```

```
 console.log([`${tenant.id}`] Coberto pelo backup global);
```

```
 break;
```

```
 case 'pitr':
```

```
 // WAL archiving + backups periódicos
```

```
 await enablePITR(tenant.id);
```

```
 break;
```

```
 case 'dedicated':
```

```
// Backup individual do banco/schema
await runFullBackup(tenant.id);
break;
}
} [text]

11. Performance

11.1 Connection Pooling por Tenant

// pool-manager.ts
import { Pool } from 'pg';
interface PoolConfig {
 max: number;
 idleTimeoutMillis: number;
 connectionTimeoutMillis: number;
}
const PLAN_POOL_LIMITS: Record<string, number> = {
 free: 2,
 pro: 10,
 enterprise: 25,
}
class PoolManager {
 private pools = new Map<string, { pool: Pool; lastUsed: number;
config: PoolConfig }>();
 getPool(tenantId: string, plan: string = 'free'): Pool {
 const existing = this.pools.get(tenantId);
 if (existing) {
 existing.lastUsed = Date.now();
 return existing.pool;
 }
 const config: PoolConfig = {
 max: PLAN_POOL_LIMITS[plan] || 5,
 idleTimeoutMillis: 30000,
```

```
 connectionTimeoutMillis: 5000,
 };
 const pool = new Pool({
 connectionString: this.getTenantDbUrl(tenantId),
 ...config,
 });
 pool.on('error', (err) => {
 console.error(Pool error for tenant ${tenantId}:, err);
 this.pools.delete(tenantId);
 });
 this.pools.set(tenantId, { pool, lastUsed: Date.now(), config });
 return pool;
}

private getTenantDbUrl(tenantId: string): string {
 // Em schema per tenant: mesma URL
 // Em DB per tenant: URL diferente por tenant
 return process.env.DATABASE_URL!;
}

async closeIdlePools(maxIdleMs: number = 300000): Promise<void>
{
 const now = Date.now();
 for (const [id, entry] of this.pools) {
 if (now - entry.lastUsed > maxIdleMs) {
 await entry.pool.end();
 this.pools.delete(id);
 console.log(Pool do tenant ${id} fechado por
inatividade);
 }
 }
}

async closeAll(): Promise<void> {
 for (const [id, entry] of this.pools) {
 await entry.pool.end();
 }
 this.pools.clear();
}
```

```
getStats(): Record<string, { totalCount: number; idleCount: number;
waitingCount: number }> {
 const stats: any = {};
 for (const [id, entry] of this.pools) {
 stats[id] = {
 totalCount: entry.pool.totalCount,
 idleCount: entry.pool.idleCount,
 waitingCount: entry.pool.waitingCount,
 };
 }
 return stats;
}
```

### ### 11.2 Query Optimization

```
// query-optimizer.ts
class QueryOptimizer {
 constructor(private tenantId: string, private plan: string) {}
 async findOrders(start: Date, end: Date): Promise<Order[]> {
 const baseQuery = `
 SELECT * FROM orders
 WHERE tenant_id = $1
 AND created_at BETWEEN $2 AND $3
 `;
 // Enterprise: busca completa
 if (this.plan === 'enterprise') {
 return pool.query(baseQuery + ' ORDER BY created_at DESC', [
 this.tenantId, start, end,
]).then(r => r.rows);
 }
 // Free/Pro: paginado
 return pool.query(baseQuery + ' ORDER BY created_at DESC
```

```
LIMIT 100', [
 this.tenantId, start, end,
]).then(r => r.rows);
}
```

```
} [text]
```

### ### 11.3 Rate Limiting por Tenant

```
// tenant-rate-limiter.ts
import { Injectable } from '@nestjs/common';
interface RateLimitConfig {
 requestsPerMinute: number;
 concurrentRequests: number;
}
const PLAN_LIMITS: Record<string, RateLimitConfig> = {
 free: { requestsPerMinute: 100, concurrentRequests: 5 },
 pro: { requestsPerMinute: 1000, concurrentRequests: 25 },
 enterprise: { requestsPerMinute: 10000, concurrentRequests: 100 },
}
@Injectable()
export class TenantRateLimiter {
 private requestCounts = new Map<string, { count: number;
windowStart: number }>();
 private concurrentCounts = new Map<string, number>();
 async checkRateLimit(tenantId: string, plan: string): Promise<void>
 {
 const limits = PLAN_LIMITS[plan] || PLAN_LIMITS.free;
 const now = Date.now();
 // const windowMs = 60_000; // 1 minuto
 // Rate limit
 const entry = this.requestCounts.get(tenantId) ?? { count: 0,
windowStart: now };
 if (now - entry.windowStart > windowMs) {
 entry.count = 0;
 entry.windowStart = now;
 }
 }
}
```

```

entry.count++;
this.requestCounts.set(tenantId, entry);
if (entry.count > limits.requestsPerMinute) {
 throw new Error(Rate limit excedido para tenant
 ${tenantId}. Limite: ${limits.requestsPerMinute}/min);
}
// Concurrent requests
const concurrent = this.concurrentCounts.get(tenantId) ?? 0;
this.concurrentCounts.set(tenantId, concurrent + 1);
if (concurrent + 1 > limits.concurrentRequests) {
 this.concurrentCounts.set(tenantId, concurrent);
 throw new Error(Muitas requisições concorrentes para
 tenant ${tenantId}. Limite: ${limits.concurrentRequests}
);
}
}
releaseConcurrent(tenantId: string): void {
 const current = this.concurrentCounts.get(tenantId) ?? 1;
 this.concurrentCounts.set(tenantId, Math.max(0, current - 1));
}
}

```

### ### 11.4 Indexação para Shared Database

```

-- Índices essenciais para shared database:
1. Sempre incluir tenant_id como primeira coluna
CREATE INDEX idx_users_tenant_id ON users (tenant_id);
CREATE INDEX idx_users_tenant_email ON users (tenant_id,
email);
CREATE INDEX idx_orders_tenant_created ON orders (tenant_id,
created_at DESC);
-- 2. Índices parciais para tenants grandes
CREATE INDEX idx_orders_acme_pending ON orders (created_at)
WHERE tenant_id = 'acme' AND status = 'pending';
-- 3. Índices funcionais para queries comuns
CREATE INDEX idx_users_tenant_lower_email ON users
(tenant_id, LOWER(email));
-- 4. Evitar índices desnecessários em colunas de baixa cardinalidade

```



-- (tenant\_id já é primeiro no índice composto)  
Query que usa o índice:

EXPLAIN ANALYZE

SELECT \* FROM users

WHERE tenant\_id = 'acme' AND email = 'joao@acme.com';

[text]

---

## ## 12. Pricing Baseado em Tenancy

### ### 12.1 Modelo de Planos

A arquitetura de isolamento escolhida define diretamente o que

| Plano      | Isolamento                   | Limites                           | Preço     | Público           |
|------------|------------------------------|-----------------------------------|-----------|-------------------|
| Free       | Shared DB                    | 5 usuários, 10 projetos, 100 MB   | Grátis    | Gratuito          |
| Pro        | Schema per Tenant            | 50 usuários, 100 projetos, 5 GB   | \$99/mês  | Profissionais     |
| Business   | Schema per Tenant (dedicado) | 200 usuários, 500 projetos, 20 GB | \$299/mês | Pequenas empresas |
| Enterprise | DB per Tenant + Réplica      | Ilimitado                         | \$499/mês | Grandes empresas  |

### ### 12.2 Feature Flags por Plano

// feature-flags.ts

```
interface PlanFeatures {
 maxUsers: number;
 maxProjects: number;
 maxStorageMB: number;
 customDomain: boolean;
 apiAccess: boolean;
 advancedReports: boolean;
 auditLog: boolean;
 prioritySupport: boolean;
 ssoEnabled: boolean;
```

```
whiteLabel: boolean;
const PLANS: Record<string, PlanFeatures> = {
 free: {
 maxUsers: 5,
 maxProjects: 10,
 maxStorageMB: 100,
 customDomain: false,
 apiAccess: false,
 advancedReports: false,
 auditLog: false,
 prioritySupport: false,
 ssoEnabled: false,
 whiteLabel: false,
 },
 pro: {
 maxUsers: 50,
 maxProjects: 100,
 maxStorageMB: 5_000,
 customDomain: false,
 apiAccess: true,
 advancedReports: true,
 auditLog: true,
 prioritySupport: false,
 ssoEnabled: false,
 whiteLabel: false,
 },
 enterprise: {
 maxUsers: Infinity,
 maxProjects: Infinity,
 maxStorageMB: Infinity,
 customDomain: true,
```

```

 apiAccess: true,
 advancedReports: true,
 auditLog: true,
 prioritySupport: true,
 ssoEnabled: true,
 whiteLabel: true,
 },
}
// feature-flag.service.ts
import { Injectable } from '@nestjs/common';
import { TenantService } from './tenant.service';
@Injectable()
export class FeatureFlagService {
 constructor(private tenantService: TenantService) {}
 private get planFeatures(): PlanFeatures {
 const plan = this.tenantService.getPlan();
 return PLANS[plan] || PLANS.free;
 }
 isFeatureEnabled(feature: keyof PlanFeatures): boolean {
 const value = this.planFeatures[feature];
 return typeof value === 'boolean' ? value : false;
 }
 getLimit(resource: 'maxUsers' | 'maxProjects' | 'maxStorageMB'):
 number {
 const value = this.planFeatures[resource];
 return typeof value === 'number' ? value : Infinity;
 }
 async checkLimit(resource: 'maxUsers' | 'maxProjects' |
'maxStorageMB'): Promise<void> {
 const limit = this.getLimit(resource);
 if (limit === Infinity) return;
 const current = await this.getCurrentUsage(resource);
 if (current >= limit) {
 throw new Error(
 Limite de ${resource} excedido

```

```

 (${current}/${limit}). Faça upgrade do plano.
);
}

```

```

private async getCurrentUsage(resource: string): Promise<number> {
 const tenantId = this.tenantService.getTenantId();
 // Consultar banco para contagem atual
 switch (resource) {
 case 'maxUsers':
 return / COUNT de users do tenant / 3;
 case 'maxProjects':
 return / COUNT de projects do tenant / 7;
 case 'maxStorageMB':
 return / SUM de tamanho de arquivos / 50;
 default:
 return 0;
 }
}

getAllFeatures(): PlanFeatures {
 return { ...this.planFeatures };
}
}

```

### ### 12.3 Guard do NestJS para Feature Flags

```

// feature.guard.ts
import { Injectable, CanActivate, ExecutionContext } from
 '@nestjs/common';
import { FeatureFlagService } from './feature-flag.service';
@Injectable()
export class FeatureGuard implements CanActivate {
 constructor(
 private featureFlagService: FeatureFlagService,

```

```

 private featureName: keyof PlanFeatures,
) {}
 canActivate(context: ExecutionContext): boolean {
 return this.featureFlagService.isFeatureEnabled(this.featureName);
 }
}
// Uso no controller
@Get('reports/advanced')
@UseGuards(new FeatureGuard('advancedReports'))
getAdvancedReports() {
 // Só executa se o plano do tenant tiver advancedReports = true
 return this.reportService.generate();
}

```

```

```

## ## 13. Testes de Isolamento entre Tenants

### ### 13.1 Configuração de Teste

```

// tenant-isolation.spec.ts
import { Test, TestingModule } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';
import * as request from 'supertest';
import { AppModule } from '../src/app.module';
import { TenantService } from '../src/tenant/tenant.service';
describe('Isolamento entre Tenants', () => {
 let app: INestApplication;
 beforeAll(async () => {
 const moduleFixture: TestingModule = await
Test.createTestingModule({
 imports: [AppModule],
})
.compile();
app = moduleFixture.createNestApplication();
await app.init();
 });
 afterAll(async () => {

```

```

 await app.close();
 });
}

// Helper para atuar como um tenant específico
const asTenant = (tenantId: string) => {
 const token = generateJwt({ sub: 'user1', tid: tenantId });
 return request(app.getHttpServer())
 .set('Authorization', Bearer ${token})
 .set('X-Tenant-Id', tenantId);
};

function generateJwt(payload: any): string {
 const jwt = require('jsonwebtoken');
 return jwt.sign(payload, process.env.JWT_SECRET || 'test-secret');
}

```

[text]

### ### 13.2 Teste 1: Vazamento Zero

```

describe('Vazamento de dados', () => {
 it('Tenant A não deve ver dados do Tenant B', async () => {
 // Arrange: criar dados como Tenant A
 await asTenant('acme')
 .post('/users')
 .send({ name: 'João Silva', email: 'joao@acme.com' });
 expect(201);
 // Act: listar usuários como Tenant B
 const response = await asTenant('zeta')
 .get('/users')
 .expect(200);
 // Assert: Tenant B não vê os dados de A
 expect(response.body).toBeInstanceOf(Array);
 expect(response.body).toHaveLength(0);
 expect(response.body).not.toContainEqual(
 expect.objectContaining({ email: 'joao@acme.com' })
);
 });
 it('deve rejeitar requisição sem tenant_id', async () => {

```

```

const jwt = generateJwt({ sub: 'user1' }); // sem tid
const response = await request(app.getHttpServer())
 .get('/users')
 .set('Authorization', Bearer ${jwt})
 .expect(401);
expect(response.body.message).toContain('Tenant');
it('deve rejeitar token com tenant inexistente', async () => {
 const response = await asTenant('tenant_inexistente')
 .get('/users')
 .expect(401);
 expect(response.body.message).toContain('não encontrado');
});

```

### ### 13.3 Teste 2: Concorrência

```

describe('Concorrência entre tenants', () => {
 it('deve processar dados de múltiplos tenants simultaneamente sem
 misturar', async () => {
 const tenants = ['acme', 'zeta', 'omega'];
 const createPromises = tenants.map((tid) =>
 asTenant(tid)
 .post('/users')
 .send({ name: User from ${tid}, email: user@${tid}.com })
 .expect(201)
);
 await Promise.all(createPromises);
 // Verificar isolamento
 for (const tid of tenants) {
 const response = await asTenant(tid).get('/users').expect(200);
 const allEmails = response.body.map((u: any) => u.email);
 const otherTenants = tenants.filter((t) => t !== tid);
 // Não deve conter emails dos outros tenants
 for (const other of otherTenants) {
 expect(allEmails).not.toContain(user@${other}.com);
 }
 }
 });
});

```

```

 }
 // Deve conter o próprio email
 expect(allEmails).toContain(user@${tid}.com);
 }
});

```

[text]

### ### 13.4 Teste 3: Injeção de Tenant ID

```

describe('Injeção de tenant_id', () => {
 it('deve ignorar tenant_id enviado no body e usar o do middleware',
 async () => {
 const response = await asTenant('acme')
 .post('/users')
 .send({
 name: 'Tentativa de invasão',
 email: 'hacker@zeta.com',
 tenant_id: 'zeta', // tentativa de criar como zeta
 })

 // expect(201);
 // Verificar que foi criado como acme, não zeta
 expect(response.body.tenant_id ||
 response.body.tenantId).toBe('acme');
 // Tentar ler como zeta
 const zetaResponse = await asTenant('zeta')
 .get('/users')

 expect(200);
 expect(zetaResponse.body).not.toContainEqual(
 expect.objectContaining({ email: 'hacker@zeta.com' })
);
 });
});

```

### ### 13.5 Teste 4: Rate Limit



```

describe('Rate limiting por plano', () => {
 it('deve bloquear tenant Free após exceder limite', async () => {
 // Tenant 'omega' é free (100 req/min no teste)
 const promises = Array.from({ length: 110 }, (_, i) =>
 asTenant('omega')
 .get('/users')
 .then((r) => r.status)
);
 const statuses = await Promise.all(promises);
 expect(tooManyRequests(statuses.filter((s) => s === 429)));
 });
 it('Enterprise não deve ser rate limited com mesma carga', async () => {
 // Tenant 'acme' é enterprise (10000 req/min)
 const promises = Array.from({ length: 110 }, () =>
 asTenant('acme')
 .get('/users')
 .then((r) => r.status)
);
 const statuses = await Promise.all(promises);
 expect(tooManyRequests(statuses.filter((s) => s === 429)));
 });
});

```

[text]

### ### 13.6 Teste 5: Migrations

```

describe('Migrations multi-tenant', () => {
 it('deve aplicar migration em todos os schemas de tenants ativos',
 async () => {
 const tenants = ['acme', 'zeta', 'omega'];
 for (const slug of tenants) {
 const result = await pool.query(`
 SELECT column_name
 FROM information_schema.columns

```

```

WHERE table_schema = 'tenant_${slug}'
AND table_name = 'users'
AND column_name = 'phone'
\}
expect(result.rows.length).toBe(1);
expect(result.rows[0].column_name).toBe('phone');
}
});

```

### ### 13.7 Teste de Quebra Proposital

```

describe('Teste de quebra (fail-safe)', () => {
 it('deve falhar se removermos o WHERE tenant_id (prova de
isolamento)', async () => {
 // Criar dados em tenants diferentes
 await asTenant('acme').post('/users').send({
 name: 'Dado sigiloso',
 email: 'secreto@acme.com',
 });
 await asTenant('zeta').post('/users').send({
 name: 'Outro dado',
 email: 'info@zeta.com',
 });
 // Query SEM filtro de tenant (simulando bug)
 const resultWithoutFilter = await pool.query('SELECT * FROM
users');
 const allUsers = resultWithoutFilter.rows;
 // Se o isolamento está funcionando, essa query retorna dados de
TODOS os tenants
 // Isso prova que sem o filtro, os dados vazam
 const tenantAcount = allUsers.filter((u: any) =>
 u.email.includes('acme')
).length;
 const tenantBcount = allUsers.filter((u: any) =>

```

```
u.email.includes('zeta')
).length;
expect(tenantAcount).toBeGreaterThan(0);
expect(tenantBcount).toBeGreaterThan(0);
// Com filtro, cada tenant vê apenas seus dados
const resultWithFilter = await pool.query(
 'SELECT * FROM users WHERE tenant_id = $1',
 ['acme']
);
expect(resultWithFilter.rows).not.toContainEqual(
 expect.objectContaining({ email: 'info@zeta.com' })
);
});
});
```